

Cuestionario de Práctica

Certificación OCP Java SE 17 Developer (1Z0-829)

295 preguntas originales tipo test, organizadas en 11 categorías según los dominios oficiales del examen.

Material de estudio elaborado con contenido propio (no reproduce texto de ningún libro con derechos de autor).

Índice de categorías

#	Categoría	Preguntas
1	1. Bloques básicos de Java (variables, tipos, texto)	26
2	2. Control de flujo (if, switch, bucles, pattern matching)	27
3	3. Programación orientada a objetos (clases, herencia, records, sealed)	26
4	4. Manejo de excepciones	27
5	5. Arrays y Collections Framework	27
6	6. Java Stream API y expresiones lambda	27
7	7. Concurrencia	27
8	8. Java I/O y NIO.2	27
9	9. JDBC (Java Database Connectivity)	27
10	10. Código seguro (secure coding)	27
11	11. Aplicaciones modulares (JPMS)	27

Instrucciones: lee cada pregunta y elige UNA sola opción correcta, salvo que se indique lo contrario. Las respuestas y explicaciones se encuentran al final del documento.

1. Bloques básicos de Java (variables, tipos, texto)

1. ¿Cuál de las siguientes es una declaración válida de una variable local usando inferencia de tipos?

- A) var x = 10;
- B) var x;
- C) final var x = null;
- D) var int x = 10;

2. ¿Qué imprime el código anterior?

```
String s = "hola";  
s.concat(" mundo");  
System.out.println(s);
```

- A) hola mundo
- B) hola
- C) mundo
- D) No compila

3. ¿Cuál es el resultado de: `var texto = """"\n Hola\n Mundo\n """"`; (bloque de texto / text block)?

- A) Se genera un String multilínea respetando la sangría mínima común
- B) Error de compilación porque var no admite text blocks
- C) El texto se guarda como una sola línea sin saltos
- D) Error porque los text blocks requieren comillas simples

4. ¿Qué imprime este código?

```
byte b = 127;  
b++;  
System.out.println(b);
```

- A) 128
- B) -128
- C) 127
- D) No compila

5. ¿Cuál de las siguientes NO es una palabra reservada de Java?

- A) record
- B) var
- C) sealed
- D) assert

6. ¿Qué imprime este código?

```
double d = 5 / 2;  
System.out.println(d);
```

- A) 2.5
- B) 2.0
- C) 2
- D) No compila

7. ¿Cuál es el propósito principal de StringBuilder frente a String?

- A) Permitir mutabilidad y evitar la creación de múltiples objetos al concatenar
- B) Hacer que las cadenas sean thread-safe automáticamente
- C) Reducir el tamaño en memoria de una cadena inmutable
- D) Permitir comparar cadenas con ==

8. ¿Qué ocurre al ejecutar este código?

```
var lista = List.of(1, 2, 3);  
lista.add(4);
```

- A) Se lanza UnsupportedOperationException en tiempo de ejecución
- B) Se agrega correctamente el elemento 4
- C) No compila
- D) Se lanza ClassCastException

9. ¿Cuál es el valor por defecto de una variable de instancia de tipo boolean?

- A) true
- B) false
- C) null
- D) 0

10. ¿Qué imprime este código?

```
int x = 10_000;  
int y = 1_0_0;  
System.out.println(x + y);
```

- A) 10100
- B) No compila
- C) 10_100
- D) Error en tiempo de ejecución

11. ¿Qué imprime este código?

```
Integer a = 127;  
Integer b = 127;  
Integer c = 200;  
Integer d = 200;  
System.out.println((a == b) + " " + (c == d));
```

- A) true true
- B) true false
- C) false true
- D) false false

12. ¿Qué imprime este código?

```
char c = 'A';  
c = (char) (c + 1);  
System.out.println(c);
```

- A) B
- B) A
- C) 66
- D) No compila

13. ¿Qué ocurre al ejecutar esta línea?

```
Integer valor = null;  
int x = valor;
```

- A) x vale 0
- B) Se lanza NullPointerException al intentar desempaquetar (unboxing) el valor null
- C) No compila
- D) x vale null

14. ¿Qué imprime este código?

```
int x = 5;  
double y = 10.0;  
System.out.println(true ? x : y);
```

- A) 5
- B) 5.0
- C) 10.0
- D) No compila

15. ¿Qué imprime este código?

```
String a = "hola";  
String b = new String("hola");  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

- A) true true
- B) true false
- C) false true
- D) false false

16. ¿Qué imprime este código?

```
String s = String.format("%05d", 42);  
System.out.println(s);
```

- A) 00042
- B) 42000
- C) 42
- D) No compila

17. ¿Cuál de las siguientes líneas NO compila?

- A) int x = 5; if (x instanceof Integer) { }
- B) Object o = 5; if (o instanceof Integer) { }
- C) String s = "a"; if (s instanceof CharSequence) { }
- D) Number n = 5; if (n instanceof Integer) { }

18. ¿Qué imprime este código?

```
int i = 5;  
int j = i++ + ++i;  
System.out.println(j);
```

- A) 10
- B) 11
- C) 12
- D) 13

19. ¿Qué imprime este código?

```
int a = 5; // 0101  
int b = 3; // 0011  
System.out.println(a & b);  
System.out.println(a | b);
```

- A) 1 7
- B) 7 1
- C) 1 1
- D) 7 7

20. ¿Qué imprime este código?

```
var arr = new int[]{1, 2, 3};  
System.out.println(arr.length);
```

- A) 3
- B) 0
- C) No compila
- D) Error en tiempo de ejecución

21. ¿Qué ocurre al ejecutar esta línea?

```
Object[] objetos = new String[3];  
objetos[0] = 10;
```

- A) Compila y se ejecuta sin problema
- B) Se lanza `ArrayStoreException` en tiempo de ejecución
- C) No compila
- D) Se lanza `ClassCastException`

22. ¿Qué imprime este código?

```
System.out.println(Character.isDigit('7') + " " + Character.isLetter('7'));
```

- A) true false
- B) false true
- C) true true
- D) false false

23. ¿Qué imprime este código?

```
System.out.println(Math.max(3, Math.abs(-7)));
```

- A) 3
- B) 7
- C) -7
- D) No compila

24. ¿Qué imprime este código?

```
String s = " Java ";  
System.out.println "[" + s.trim() + "]" );
```

- A) [Java]
- B) [Java]
- C) [Java]
- D) No compila

25. ¿Qué imprime este código?

```
int[] arr = {1, 2, 3};  
for (int n : arr) {  
    n = n * 10;  
}  
System.out.println(Arrays.toString(arr));
```

- A) [1, 2, 3]
- B) [10, 20, 30]
- C) [10, 2, 3]
- D) No compila

26. ¿En cuál de los siguientes contextos NO se puede usar 'var'?

- A) Como tipo de un parámetro de un método regular (no lambda)
- B) En una variable local con inicializador
- C) Dentro de un bloque try-with-resources
- D) Como variable de control de un bucle for tradicional

2. Control de flujo (if, switch, bucles, pattern matching)

27. ¿Qué imprime el código anterior?

```
int x = 2;
String resultado = switch (x) {
    case 1 -> "uno";
    case 2 -> "dos";
    default -> "otro";
};
System.out.println(resultado);
```

- A) uno
- B) dos
- C) otro
- D) No compila

28. ¿Qué imprime este código (pattern matching for instanceof)?

```
Object obj = "texto";
if (obj instanceof String s && s.length() > 3) {
    System.out.println(s.toUpperCase());
} else {
    System.out.println("no aplica");
}
```

- A) TEXTO
- B) no aplica
- C) texto
- D) No compila

29. ¿Qué imprime este código?

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue;
        System.out.print(i + " " + j);
    }
}
```

- A) 001122
- B) 002021220
- C) 0021 1122
- D) 020212 22

30. ¿Qué imprime este switch clásico (con dos puntos)?

```
int y = 10;
switch (y) {
    case 10:
        System.out.println("diez");
    case 20:
        System.out.println("veinte");
        break;
    default:
        System.out.println("otro");
}
```

- A) diez
- B) diez\nveinte
- C) veinte
- D) otro

31. ¿Cuál de las siguientes etiquetas de switch es válida para un patrón de tipo (type pattern) en Java 17 (preview / posterior)?

- A) case Integer i -> System.out.println(i);
- B) case int i -> System.out.println(i);
- C) case (Integer) i -> System.out.println(i);
- D) case Integer: i -> System.out.println(i);

32. ¿Qué imprime este código con etiqueta (labeled break)?

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) break outer;
        System.out.print(i + " " + j + " ");
    }
}
```

- A) 00
- B) 00 01
- C) 00 10 20
- D) No imprime nada

33. ¿Qué imprime este bucle do-while?

```
int n = 5;
do {
    System.out.print(n + " ");
    n--;
} while (n > 5);
```

- A) 5 4 3 2 1
- B) 5
- C) Bucle infinito
- D) No imprime nada

34. En un switch de tipo expresión (switch expression) que usa 'yield', ¿en qué contexto se utiliza?

- A) Dentro de un bloque { } de un case, para devolver el valor de la expresión switch
- B) Para pausar la ejecución del hilo actual
- C) Solo puede usarse en bucles for-each
- D) Es sinónimo de 'return' en cualquier método

35. ¿Qué imprime este código?

```
for (var i = 0; i < 5; i += 2) {  
    if (i == 4) break;  
    System.out.print(i);  
}
```

- A) 024
- B) 02
- C) 0246
- D) No compila

36. ¿Qué sucede si un bloque switch expression con 'case ->' no cubre todos los valores posibles de un enum y no tiene 'default'?

- A) Compila sin problema siempre
- B) El compilador exige exhaustividad y puede dar error si faltan casos y no hay default
- C) Se lanza NullPointerException en tiempo de ejecución siempre
- D) Se ignora silenciosamente el caso faltante

37. ¿Qué imprime este código?

```
String dia = "MAR";  
switch (dia) {  
    case "LUN": System.out.print("L"); break;  
    case "MAR": System.out.print("M"); break;  
    default: System.out.print("?");  
}
```

- A) L
- B) M
- C) ?
- D) No compila

38. ¿Qué imprime este código con ternarios anidados?

```
int x = 5;  
String r = x > 10 ? "grande" : x > 3 ? "medio" : "pequeño";  
System.out.println(r);
```

- A) grande
- B) medio
- C) pequeño
- D) No compila

39. ¿Compila este código?

```
int x = switch(1) {  
    case 1: break 5;  
    default: yield 0;  
};
```

- A) Sí, sin problema
- B) No, 'break' no puede devolver un valor en un switch expression; se debe usar 'yield'
- C) No, falta un case adicional
- D) No, un switch expression no admite 'case:' con dos puntos

40. ¿Qué imprime este código?

```
int i = 0;  
while (true) {  
    i++;  
    if (i == 3) break;  
}  
System.out.println(i);
```

- A) 3
- B) 2
- C) 4
- D) Bucle infinito

41. ¿Qué imprime este código?

```
Map<String, Integer> m = new LinkedHashMap<>();  
m.put("a", 1);  
m.put("b", 2);  
for (Map.Entry<String, Integer> e : m.entrySet()) {  
    System.out.print(e.getKey());  
}
```

- A) ab
- B) ba
- C) {a=1, b=2}
- D) No compila

42. ¿Compila este código si 'color' es de tipo Color?

```
enum Color { ROJO, VERDE, AZUL }  
// suponiendo 'color' de tipo Color  
String s = switch (color) {  
    case ROJO -> "r";  
    case VERDE -> "v";  
};
```

- A) Sí, siempre
- B) No, falta cubrir AZUL o agregar un caso default
- C) Sí, pero falla en tiempo de ejecución
- D) No, switch expression no admite valores enum

43. ¿Qué imprime este código (do-while sin llaves)?

```
int i = 0;
do
    System.out.print(i);
while (++i < 3);
```

- A) 012
- B) 0123
- C) Bucle infinito
- D) No compila

44. ¿Qué imprime este código?

```
int[] arr = null;
if (arr != null && arr.length > 0) {
    System.out.println("tiene elementos");
} else {
    System.out.println("vacio o null");
}
```

- A) tiene elementos
- B) vacio o null
- C) NullPointerException
- D) No compila

45. ¿Qué imprime este código con etiqueta (continue outer)?

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;
        System.out.print(i + " " + j);
    }
}
```

- A) 001020
- B) 000102
- C) 00 10 20
- D) Bucle infinito

46. ¿Qué imprime este código con etiquetas de case agrupadas?

```
int x = 2;
switch (x) {
    case 1:
    case 2:
        System.out.print("uno o dos");
        break;
    default:
        System.out.print("otro");
}
```

- A) uno o dos
- B) otro
- C) unodos
- D) No compila

47. ¿Qué imprime este bucle for con condición pero sin actualización explícita?

```
int i = 0;
for (; i < 5; ) {
    System.out.print(i);
    i += 2;
}
```

- A) 024
- B) 01234
- C) 02 4
- D) Bucle infinito

48. ¿Qué imprime este código?

```
var resultado = switch (3) {
    case 1, 2 -> "bajo";
    case 3, 4 -> "medio";
    default -> "alto";
};
System.out.println(resultado);
```

- A) bajo
- B) medio
- C) alto
- D) No compila

49. ¿Qué imprime este bucle 'for' infinito con break inmediato?

```
for (;;) {
    System.out.print("x");
    break;
}
```

- A) Bucle infinito, nunca termina
- B) x
- C) No compila
- D) Error en tiempo de compilación

50. ¿Qué imprime este código (dangling else)?

```
int x = 5;
if (x > 0)
    if (x > 10)
        System.out.print("grande");
    else
        System.out.print("pequeño");
```

- A) grande
- B) pequeño
- C) No imprime nada
- D) No compila

51. ¿Qué imprime este código?

```
byte b = 10;
b += 5;
System.out.println(b);
```

- A) 15
- B) No compila
- C) Error en tiempo de ejecución
- D) 10

52. ¿Qué ocurre al ejecutar esta línea?

```
Boolean flag = null;
int x = flag ? 1 : 2;
```

- A) x vale 2
- B) Se lanza NullPointerException al desempaquetar flag
- C) No compila
- D) x vale 1

53. ¿Qué imprime este código?

```
String s = "b";
switch (s) {
    default:
        System.out.print("def-");
    case "a":
        System.out.print("a");
        break;
    case "b":
        System.out.print("b");
}
```

- A) b
- B) a
- C) def-a
- D) No compila

3. Programación orientada a objetos (clases, herencia, records, sealed)

54. ¿Qué tipo de constructor se ha definido dentro del record 'Punto'?

```
public record Punto(int x, int y) {  
    public Punto {  
        if (x < 0) throw new IllegalArgumentException("x negativo");  
    }  
}
```

- A) Un constructor compacto (compact constructor)
- B) Un constructor sobrecargado normal
- C) Un constructor estático
- D) No compila porque falta la lista de parámetros

55. ¿Qué garantiza el uso de 'sealed' e 'permits' en este código?

```
public sealed interface Figura permits Circulo, Cuadrado {}  
public final class Circulo implements Figura {}  
public final class Cuadrado implements Figura {}
```

- A) Que solo las clases listadas pueden implementar la interfaz
- B) Que la interfaz no puede tener métodos abstractos
- C) Que las clases hijas deben ser todas abstractas
- D) Que se genera automáticamente un método equals()

56. ¿Qué imprime este código?

```
class Animal {  
    void hacerSonido() { System.out.println("..."); }  
}  
class Perro extends Animal {  
    @Override  
    void hacerSonido() { System.out.println("Guau"); }  
}  
Animal a = new Perro();  
a.hacerSonido();
```

- A) ...
- B) Guau
- C) No compila
- D) Error en tiempo de ejecución

57. ¿Cuál de las siguientes NO es una característica automática de un 'record'?

- A) Métodos de acceso (accessors) con el mismo nombre que los campos
- B) Implementación de equals() y hashCode() basada en los campos
- C) Un método toString() legible
- D) Posibilidad de heredar de otra clase concreta

58. ¿Qué ocurre al compilar este código?

```
abstract class Vehiculo {  
    abstract void mover();  
}  
class Auto extends Vehiculo {  
    // sin implementar mover()  
}
```

- A) Compila correctamente
- B) No compila: Auto debe implementar mover() o ser declarada abstracta también
- C) Compila pero lanza excepción en tiempo de ejecución al instanciar Auto
- D) No compila porque Vehiculo no tiene constructor

59. ¿Qué modificador de acceso permite que un miembro sea visible solo dentro del mismo paquete y por subclases (incluso en otro paquete)?

- A) private
- B) protected
- C) default (sin modificador)
- D) public

60. ¿Qué debe hacer la clase Mixto para compilar correctamente?

```
interface Saludo {  
    default String decir() { return "Hola"; }  
}  
interface Despedida {  
    default String decir() { return "Adiós"; }  
}  
class Mixto implements Saludo, Despedida {  
    // ¿Qué se necesita aquí?  
}
```

- A) Nada, Java elige automáticamente el método de Saludo
- B) Sobrescribir el método decir() para resolver el conflicto explícitamente
- C) No es posible implementar dos interfaces con métodos default con igual firma
- D) Usar 'super.decir()' sin sobrescribir el método

61. ¿Cuál es el propósito de un bloque de inicialización de instancia (instance initializer block) en una clase?

- A) Ejecutar código antes de cada constructor, útil para lógica común de inicialización
- B) Reemplazar por completo la necesidad de constructores
- C) Ejecutarse solo una vez por clase, sin importar cuántas instancias se creen
- D) Definir métodos estáticos

62. ¿Este código compila correctamente?

```
enum Dia {  
    LUNES, MARTES, MIERCOLES;  
    public String describir() {  
        return switch (this) {  
            case LUNES -> "Inicio";  
            case MARTES, MIERCOLES -> "Medio";  
        };  
    }  
}
```

- A) Sí, es exhaustivo porque cubre los tres valores del enum
- B) No, falta un caso default obligatorio
- C) No, un enum no puede tener métodos
- D) No, switch no admite múltiples etiquetas separadas por coma

63. ¿Qué keyword se usa para permitir que una subclase de una clase 'sealed' sea extendida libremente por cualquier otra clase?

- A) open
- B) non-sealed
- C) unsealed
- D) extends-all

64. ¿Qué imprime este código?

```
class Persona {  
    String nombre;  
    Persona(String n) { nombre = n; }  
}  
Persona p1 = new Persona("Ana");  
Persona p2 = new Persona("Ana");  
System.out.println(p1.equals(p2));
```

- A) true
- B) false
- C) No compila
- D) Error en tiempo de ejecución

65. ¿Qué imprime este código?

```
class A { static String id() { return "A"; } }  
class B extends A { static String id() { return "B"; } }  
A obj = new B();  
System.out.println(obj.id());
```

- A) A
- B) B
- C) No compila
- D) Error en tiempo de ejecución

66. ¿Qué imprime este código?

```
abstract class Figura {  
    Figura() { System.out.println("Figura creada"); }  
}  
class Circulo extends Figura {  
    Circulo() { super(); System.out.println("Circulo creado"); }  
}  
new Circulo();
```

- A) Circulo creado
- B) Figura creada seguido de Circulo creado
- C) No compila porque Figura es abstracta
- D) Error en tiempo de ejecución

67. ¿Qué imprime este código?

```
interface Utilidades {  
    static int doble(int x) { return x * 2; }  
}  
System.out.println(Utilidades.doble(5));
```

- A) 10
- B) No compila porque las interfaces no pueden tener métodos estáticos
- C) Error en tiempo de ejecución
- D) 5

68. ¿Es válido este código (Java 9+)?

```
interface Calculadora {  
    private int ayuda() { return 1; }  
    default int calcular() { return ayuda() + 1; }  
}
```

- A) Sí, los métodos privados de interfaz permiten reutilizar lógica entre métodos default/estáticos
- B) No, las interfaces no pueden tener métodos privados
- C) No, los métodos default no pueden invocar métodos privados
- D) No compila porque falta 'abstract'

69. ¿Es válido este código?

```
interface Nombrable { String nombre(); }  
record Persona(String nombre, int edad) implements Nombrable {}
```

- A) Sí, compila correctamente: el accessor automático nombre() satisface el método de la interfaz
- B) No, los records no pueden implementar interfaces
- C) No, falta implementar el método edad()
- D) Error en tiempo de ejecución al crear una instancia

70. ¿Es correcta esta forma de instanciar la clase interna (no estática)?

```
class Externa {  
    class Interna {}  
}  
Externa ext = new Externa();  
Externa.Interna in = ext.new Interna();
```

- A) Sí, es la sintaxis correcta para una inner class no estática
- B) No, se debe usar new Externa.Interna()
- C) No, las inner classes no pueden instanciarse desde fuera
- D) No compila nunca

71. ¿Qué característica tiene 'Nido' al ser una clase anidada estática?

```
class Externa {  
    static class Nido {}  
}  
Externa.Nido n = new Externa.Nido();
```

- A) No necesita una instancia de Externa para ser creada
- B) Siempre requiere una instancia de Externa
- C) No puede tener métodos
- D) No puede ser pública

72. ¿Qué imprime este código?

```
interface Sonido { String emitir(); }  
enum Animal implements Sonido {  
    PERRO { public String emitir() { return "Guau"; } },  
    GATO { public String emitir() { return "Miau"; } };  
}  
System.out.println(Animal.GATO.emitir());
```

- A) Miau
- B) Guau
- C) No compila
- D) GATO

73. ¿Qué imprime este código?

```
class A {  
    void metodo(int x) { System.out.println("int"); }  
}  
class B extends A {  
    void metodo(long x) { System.out.println("long"); }  
}  
new B().metodo(5);
```

- A) int
- B) long
- C) No compila
- D) Ambos

74. ¿Qué ocurre al intentar compilar la clase B?

```
class A {  
    final void mostrar() { System.out.println("A"); }  
}  
class B extends A {  
    void mostrar() { System.out.println("B"); }  
}
```

- A) Compila y sobrescribe mostrar()
- B) No compila: no se puede sobrescribir un método final
- C) Error en tiempo de ejecución
- D) Compila, pero mostrar() de B nunca se ejecuta

75. ¿Qué principio demuestra el uso de 'this(0, 0)' en el constructor sin argumentos?

```
class Punto {  
    int x, y;  
    Punto() { this(0, 0); }  
    Punto(int x, int y) { this.x = x; this.y = y; }  
}  
new Punto();
```

- A) Encadenamiento de constructores (constructor chaining) para reutilizar lógica de inicialización
- B) Sobrecarga de operadores
- C) Anulación de método (overriding)
- D) Inicialización estática

76. ¿Cuántas veces se imprime "bloque estático"?

```
class Config {  
    static int valor;  
    static {  
        valor = 10;  
        System.out.println("bloque estático");  
    }  
}  
new Config();  
new Config();
```

- A) 1
- B) 2
- C) 0
- D) Depende del recolector de basura

77. ¿Por qué compila esta sobrescritura aunque el tipo de retorno sea distinto (Perro en vez de Animal)?

```
class Animal {  
    Animal reproducir() { return new Animal(); }  
}  
class Perro extends Animal {  
    @Override  
    Perro reproducir() { return new Perro(); }  
}
```

- A) Porque Java permite tipos de retorno covariantes: Perro es un subtipo de Animal
- B) Porque los tipos de retorno se ignoran al sobrescribir
- C) Porque @Override lo permite sin restricciones
- D) No debería compilar, es un error

78. ¿Qué imprime este código?

```
class Producto {  
    String nombre;  
    Producto(String n) { nombre = n; }  
    @Override  
    public String toString() { return "Producto[" + nombre + "]; }  
}  
System.out.println(new Producto("Mesa"));
```

- A) Producto[Mesa]
- B) Producto@hash
- C) Mesa
- D) No compila

79. ¿Por qué puede este método acceder directamente a 'otra.saldo' aunque sea un campo privado?

```
class Cuenta {  
    private double saldo;  
    Cuenta(double s) { saldo = s; }  
    boolean tieneMasSaldoQue(Cuenta otra) {  
        return this.saldo > otra.saldo;  
    }  
}
```

- A) Porque el acceso 'private' se aplica a nivel de CLASE, no de instancia: cualquier método de Cuenta puede acceder a los campos privados de otro objeto Cuenta
- B) Porque 'saldo' no es realmente privado en este caso
- C) No compila, es un error de acceso
- D) Porque 'otra' es un parámetro especial reservado

4. Manejo de excepciones

80. ¿Qué imprime este código?

```
try {  
    throw new RuntimeException("A");  
} catch (RuntimeException e) {  
    System.out.print("B");  
} finally {  
    System.out.print("C");  
}  
System.out.print("D");
```

- A) BC
- B) BCD
- C) CD
- D) ABCD

81. ¿Qué problema tiene este código?

```
public class MiExcepcion extends Exception {  
    public MiExcepcion(String msg) { super(msg); }  
}  
  
public void metodo() {  
    throw new MiExcepcion("error");  
}
```

- A) MiExcepcion extiende Exception (checked), por lo que 'metodo' debe declarar 'throws MiExcepcion' o capturarla
- B) No se puede extender Exception directamente
- C) El constructor no puede llamar a super(msg)
- D) No hay ningún problema, compila perfectamente

82. Para que este 'try-with-resources' compile, ¿qué debe implementar MiRecurso?

```
try (var recurso = new MiRecurso()) {  
    recurso.usar();  
} catch (Exception e) {  
    System.out.println("Error");  
}
```

- A) Cloneable
- B) AutoCloseable
- C) Serializable
- D) Comparable

83. ¿Qué imprime este código?

```
try {  
    int[] arr = new int[2];  
    System.out.println(arr[5]);  
} catch (NullPointerException e) {  
    System.out.println("NPE");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("AIOOBE");  
}
```

- A) NPE
- B) AIOOBE
- C) No compila
- D) No se captura la excepción

84. ¿Cuál de estas excepciones es 'unchecked' (no verificada por el compilador)?

- A) IOException
- B) SQLException
- C) NumberFormatException
- D) InterruptedException

85. ¿Este código compila?

```
try {  
    throw new IllegalStateException("x");  
} catch (RuntimeException | Error e) {  
    System.out.println("capturado");  
} finally {  
    return;  
}
```

- A) Sí, multi-catch con RuntimeException y Error es válido
- B) No, no se puede combinar RuntimeException y Error en un multi-catch
- C) No, finally no puede tener un return
- D) No, falta declarar throws en el método

86. ¿Qué devuelve el método calcular()?

```
public int calcular() {  
    try {  
        return 1;  
    } finally {  
        return 2;  
    }  
}
```

- A) 1
- B) 2
- C) No compila
- D) Lanza excepción en tiempo de ejecución

87. En try-with-resources con múltiples recursos declarados, ¿en qué orden se cierran?

- A) En el mismo orden en que fueron declarados
- B) En orden inverso al de su declaración
- C) En orden alfabético del nombre de la variable
- D) El orden no está definido

88. ¿Qué problema tiene este fragmento (suponiendo un try previo válido)?

```
catch (Exception e) {  
    System.out.println(e.getMessage());  
} catch (ArithmeticException ae) {  
    System.out.println("aritmética");  
}
```

- A) No compila: el catch de ArithmeticException es inalcanzable porque Exception ya lo cubre y está antes
- B) No hay ningún problema
- C) Falta un finally obligatorio
- D) ArithmeticException no puede capturarse nunca

89. ¿Cuál es la superclase común de Exception y Error?

- A) RuntimeException
- B) Throwable
- C) Object
- D) IOException

90. ¿Qué imprime este código?

```
class ServicioException extends Exception {  
    public ServicioException(String msg, Throwable causa) { super(msg, causa); }  
}  
try {  
    try {  
        throw new NullPointerException("original");  
    } catch (NullPointerException e) {  
        throw new ServicioException("fallo servicio", e);  
    }  
} catch (ServicioException se) {  
    System.out.println(se.getCause().getMessage());  
}
```

- A) original
- B) fallo servicio
- C) null
- D) No compila

91. ¿Por qué compila 'throws IOException, SQLException' aunque se capture y relance un 'Exception' genérico?

```
void metodo() throws IOException, SQLException {  
    try {  
        if (Math.random() > 2) throw new IOException();  
        else throw new SQLException();  
    } catch (Exception e) {  
        throw e;  
    }  
}
```

- A) Por el análisis de 'rethrow preciso' (precise rethrow) desde Java 7, que infiere los tipos reales lanzados dentro del try
- B) Porque Exception siempre se traduce automáticamente a esos tipos
- C) No compila; se necesitaría 'throws Exception'
- D) Porque IOException y SQLException no son checked

92. Si las aserciones NO están habilitadas (comportamiento por defecto de la JVM), ¿qué imprime este código?

```
int x = -5;  
assert x > 0 : "x debe ser positivo";  
System.out.println("fin");
```

- A) fin
- B) Se lanza AssertionError
- C) No compila
- D) x debe ser positivo

93. ¿Es válido un bloque try sin catch, solo con finally?

```
try {  
    System.out.println("intento");  
} finally {  
    System.out.println("finally");  
}
```

- A) Sí, es completamente válido
- B) No, siempre se requiere al menos un catch
- C) No compila nunca
- D) Solo es válido en try-with-resources

94. ¿Qué imprime este código?

```
class Recurso {  
    Recurso() { throw new RuntimeException("fallo al crear"); }  
}  
try {  
    Recurso r = new Recurso();  
} catch (RuntimeException e) {  
    System.out.println("capturado: " + e.getMessage());  
}
```

- A) capturado: fallo al crear
- B) No compila
- C) r se crea con valor null
- D) Error en tiempo de ejecución no capturado

95. ¿Qué error se produce al invocar recursivo()?

```
void recursivo() { recursivo(); }
```

- A) StackOverflowError
- B) OutOfMemoryError
- C) IllegalStateException
- D) No ocurre ningún error, se ejecuta indefinidamente

96. ¿Qué imprime este código?

```
try {  
    int x = Integer.parseInt("abc");  
} catch (NumberFormatException e) {  
    System.out.println("no numérico");  
}
```

- A) no numérico
- B) abc
- C) 0
- D) No compila

97. ¿Qué ocurre al ejecutar esta línea?

```
Object o = "texto";  
Integer i = (Integer) o;
```

- A) i vale null
- B) Se lanza ClassCastException en tiempo de ejecución
- C) No compila
- D) i vale 0

98. ¿Por qué esta excepción NO necesita declararse en la cláusula 'throws' del método que la lanza?

```
class SaldoInsuficienteException extends RuntimeException {  
    public SaldoInsuficienteException() { super(); }  
    public SaldoInsuficienteException(String msg) { super(msg); }  
}  
throw new SaldoInsuficienteException();
```

- A) Porque extiende RuntimeException, que es una excepción 'unchecked'
- B) Porque tiene múltiples constructores
- C) Porque no tiene mensaje en uno de sus constructores
- D) No es cierto: sí debe declararse siempre

99. Si tanto el cuerpo del try como close() lanzan excepciones, ¿cuál se propaga como excepción principal?

```
class Recurso implements AutoCloseable {  
    public void close() { throw new RuntimeException("error al cerrar"); }  
}  
try (Recurso r = new Recurso()) {  
    throw new RuntimeException("error en el cuerpo");  
}
```

- A) La del cuerpo del try; la de close() queda registrada como excepción suprimida (suppressed)
- B) La de close(), ignorando la del cuerpo
- C) Ambas se lanzan simultáneamente como excepciones separadas
- D) Ninguna; se ignoran silenciosamente

100. ¿Qué excepción se propaga finalmente al llamador?

```
try {  
    throw new RuntimeException("original");  
} finally {  
    throw new IllegalStateException("del finally");  
}
```

- A) RuntimeException("original")
- B) IllegalStateException("del finally"); la original se pierde
- C) Ambas encadenadas automáticamente
- D) No compila

101. ¿Compila este código?

```
Runnable r = () -> {  
    throw new IOException("fallo");  
};
```

- A) Sí, sin problema
- B) No, porque Runnable.run() no declara 'throws IOException' y esta es checked
- C) No, porque las lambdas no pueden lanzar excepciones
- D) Sí, pero solo si se usa Callable en lugar de Runnable

102. ¿Es válido este try-with-resources usando una variable ya existente (Java 9+)?

```
MiRecurso recurso = new MiRecurso();
try (recurso) {
    recurso.usar();
}
```

- A) Sí, siempre que 'recurso' sea final o efectivamente final
- B) No, el recurso debe declararse dentro del paréntesis del try
- C) No compila en ninguna versión de Java
- D) Solo es válido si recurso es null

103. ¿Qué ocurre al ejecutar lista.add(4)?

```
Integer[] arr = {1, 2, 3};
List<Integer> lista = Arrays.asList(arr);
lista.add(4);
```

- A) Se agrega correctamente
- B) Se lanza UnsupportedOperationException porque Arrays.asList() devuelve una lista de tamaño fijo
- C) Se lanza ArrayIndexOutOfBoundsException
- D) No compila

104. ¿Qué ocurre al invocar procesar(null)?

```
public void procesar(String dato) {
    Objects.requireNonNull(dato, "dato no puede ser null");
    System.out.println(dato.length());
}
procesar(null);
```

- A) Se imprime 0
- B) Se lanza NullPointerException con el mensaje 'dato no puede ser null'
- C) No compila
- D) Se imprime 'null'

105. ¿Es válido capturar un Error usando 'catch (Throwable t)'?

```
try {
    throw new Error("error grave");
} catch (Throwable t) {
    System.out.println("capturado: " + t.getMessage());
}
```

- A) Sí, Throwable es superclase tanto de Exception como de Error
- B) No, los Error nunca pueden capturarse
- C) No compila
- D) Solo se puede capturar con catch (Error e)

106. ¿Por qué esta línea dentro del catch NO compila?

```
try {  
    riesgoso();  
} catch (IOException | SQLException e) {  
    e = new IOException(); // ¿Es válido?  
}
```

- A) Porque en un multi-catch, la variable de excepción es implícitamente final y no puede reasignarse
- B) Porque IOException no puede asignarse a una variable
- C) Porque falta declarar 'throws' en el método
- D) Sí compila sin problema

5. Arrays y Collections Framework

107. ¿Qué imprime este código?

```
int[] arr = {5, 3, 8, 1};  
Arrays.sort(arr);  
System.out.println(Arrays.toString(arr));
```

- A) [5, 3, 8, 1]
- B) [1, 3, 5, 8]
- C) [8, 5, 3, 1]
- D) No compila

108. ¿Qué ocurre al ejecutar este código?

```
List<String> lista = new ArrayList<>(List.of("a", "b", "c"));  
Iterator<String> it = lista.iterator();  
while (it.hasNext()) {  
    String s = it.next();  
    if (s.equals("b")) lista.remove(s);  
}
```

- A) Se elimina 'b' correctamente sin problema
- B) Se lanza ConcurrentModificationException
- C) Se lanza NoSuchElementException
- D) No compila

109. ¿Qué imprime este código?

```
Map<String, Integer> mapa = new HashMap<>();  
mapa.put("a", 1);  
mapa.merge("a", 10, Integer::sum);  
System.out.println(mapa.get("a"));
```

- A) 1
- B) 10
- C) 11
- D) No compila

110. ¿Cuál de las siguientes implementaciones de List mantiene el orden de inserción y permite acceso rápido por índice ($O(1)$)?

- A) LinkedList
- B) ArrayList
- C) TreeSet
- D) HashSet

111. ¿Qué imprime este código?

```
Set<String> set = new TreeSet<>();  
set.add("banana");  
set.add("apple");  
set.add("cherry");  
System.out.println(set);
```

- A) [banana, apple, cherry]
- B) [apple, banana, cherry]
- C) [cherry, banana, apple]
- D) Orden no determinístico

112. ¿Qué imprime este código?

```
var deque = new ArrayDeque<Integer>();  
deque.addFirst(1);  
deque.addLast(2);  
deque.addFirst(3);  
System.out.println(deque);
```

- A) [3, 1, 2]
- B) [1, 2, 3]
- C) [2, 1, 3]
- D) [3, 2, 1]

113. ¿Qué imprime este código?

```
List<Integer> nums = new ArrayList<>(List.of(3, 1, 2));  
nums.sort(Comparator.reverseOrder());  
System.out.println(nums);
```

- A) [1, 2, 3]
- B) [3, 2, 1]
- C) [3, 1, 2]
- D) No compila

114. ¿Qué método de Collections se usa para obtener una vista de solo lectura de una lista mutable?

- A) Collections.unmodifiableList(lista)
- B) Collections.readOnly(lista)
- C) lista.toImmutableList()
- D) Collections.freeze(lista)

115. ¿Qué imprime este código?

```
int[][] matriz = new int[2][3];  
System.out.println(matriz.length + " " + matriz[0].length);
```

- A) 2 3
- B) 3 2
- C) 6 6
- D) No compila

116. ¿Cuál es la diferencia principal entre HashMap y LinkedHashMap?

- A) LinkedHashMap no permite valores nulos
- B) LinkedHashMap mantiene el orden de inserción de las claves, HashMap no garantiza ningún orden
- C) HashMap es más rápido en todos los casos
- D) LinkedHashMap no permite claves duplicadas y HashMap sí

117. ¿Qué imprime este código?

```
var lista = new ArrayList<Integer>(List.of(1, 2, 3, 4, 5));  
lista.removeIf(n -> n % 2 == 0);  
System.out.println(lista);
```

- A) [1, 3, 5]
- B) [2, 4]
- C) [1, 2, 3, 4, 5]
- D) No compila

118. ¿Qué imprime este código?

```
Queue<Integer> pq = new PriorityQueue<>();  
pq.add(5); pq.add(1); pq.add(3);  
System.out.println(pq.poll());
```

- A) 5
- B) 1
- C) 3
- D) No compila

119. ¿Qué imprime este código?

```
List<Integer> nums = List.of(4, 1, 9, 2);  
System.out.println(Collections.max(nums) + " " + Collections.min(nums));
```

- A) 9 1
- B) 1 9
- C) 4 2
- D) No compila

120. ¿Qué ocurre al ejecutar copia.add(4)?

```
List<Integer> original = new ArrayList<>(List.of(1, 2, 3));  
List<Integer> copia = List.copyOf(original);  
copia.add(4);
```

- A) Se agrega sin problema
- B) Se lanza UnsupportedOperationException, porque List.copyOf() es immutable
- C) Se modifica también 'original'
- D) No compila

121. ¿Qué imprime este código?

```
List<String> nombres = new ArrayList<>(List.of("Ana", "Bo", "Al"));
nombres.sort(Comparator.comparing(String::length).thenComparing(Comparator.naturalOrder()));
;
System.out.println(nombres);
```

- A) [Bo, Al, Ana]
- B) [Al, Bo, Ana]
- C) [Ana, Al, Bo]
- D) No compila

122. ¿Qué imprime este código?

```
TreeSet<Integer> set = new TreeSet<>(Set.of(1, 3, 5, 7));
System.out.println(set.floor(4) + " " + set.ceiling(4));
```

- A) 3 5
- B) 5 3
- C) 4 4
- D) No compila

123. ¿Qué imprime este código?

```
Map<String, Integer> m = Map.of("a", 1, "b", 2, "c", 3);
int suma = m.values().stream().mapToInt(Integer::intValue).sum();
System.out.println(suma);
```

- A) 6
- B) 3
- C) 0
- D) No compila

124. ¿Qué imprime este código?

```
List<String> lista = new ArrayList<>(List.of("banana", "kiwi", "fresa"));
Collections.sort(lista, Comparator.comparing(String::length));
System.out.println(lista);
```

- A) [kiwi, fresa, banana]
- B) [banana, fresa, kiwi]
- C) [fresa, kiwi, banana]
- D) No compila

125. ¿Qué imprime este código?

```
class Punto {  
    int x, y;  
    Punto(int x, int y) { this.x = x; this.y = y; }  
}  
Set<Punto> set = new HashSet<>();  
set.add(new Punto(1, 1));  
set.add(new Punto(1, 1));  
System.out.println(set.size());
```

- A) 1
- B) 2
- C) 0
- D) No compila

126. ¿Qué imprime este código?

```
List<Integer> lista = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
List<Integer> sub = lista.subList(1, 3);  
sub.set(0, 99);  
System.out.println(lista);
```

- A) [1, 2, 3, 4, 5]
- B) [1, 99, 3, 4, 5]
- C) [99, 2, 3, 4, 5]
- D) No compila

127. ¿Qué imprime este código?

```
Integer[] arr = {1, 2, 3};  
List<Integer> lista = Arrays.asList(arr);  
arr[0] = 99;  
System.out.println(lista.get(0));
```

- A) 1
- B) 99
- C) No compila
- D) Error en tiempo de ejecución

128. ¿Qué imprime este código?

```
Stack<Integer> pila = new Stack<>();  
pila.push(1); pila.push(2); pila.push(3);  
System.out.println(pila.pop() + " " + pila.peek());
```

- A) 3 2
- B) 3 3
- C) 1 2
- D) 2 1

129. ¿Qué característica distingue a Vector de ArrayList?

- A) Vector está sincronizado internamente (thread-safe), con menor rendimiento en entornos de un solo hilo
- B) ArrayList está sincronizado y Vector no
- C) No hay ninguna diferencia real entre ambos
- D) Vector no permite elementos duplicados

130. ¿Qué imprime este código?

```
enum Dia { LUN, MAR, MIE, JUE, VIE }  
EnumSet<Dia> laborables = EnumSet.range(Dia.LUN, Dia.VIE);  
System.out.println(laborables.size());
```

- A) 5
- B) 4
- C) 0
- D) No compila

131. ¿Qué ocurre al ejecutar vacia.add("x")?

```
List<String> vacia = Collections.emptyList();  
vacía.add("x");
```

- A) Se agrega el elemento
- B) Se lanza UnsupportedOperationException
- C) Se lanza NullPointerException
- D) No compila

132. ¿Qué imprime este código?

```
List<String> lista = List.of("a", "b", "c", "b");  
System.out.println(lista.indexOf("b"));
```

- A) 1
- B) 3
- C) -1
- D) No compila

133. ¿Por qué el resultado de esta búsqueda binaria puede ser incorrecto o impredecible?

```
List<Integer> lista = new ArrayList<>(List.of(5, 1, 3));  
int pos = Collections.binarySearch(lista, 3);
```

- A) Porque la lista no está ordenada previamente, y binarySearch requiere una lista ordenada para funcionar correctamente
- B) Porque 3 no existe en la lista
- C) Porque binarySearch no funciona con Integer
- D) No compila

6. Java Stream API y expresiones lambda

134. ¿Qué imprime este código?

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);  
long total = nums.stream()  
    .filter(n -> n % 2 == 0)  
    .count();  
System.out.println(total);
```

- A) 2
- B) 3
- C) 5
- D) No compila

135. ¿Qué ocurre al ejecutar este código?

```
Stream<Integer> s = Stream.of(1, 2, 3);  
s.forEach(System.out::println);  
s.forEach(System.out::println);
```

- A) Imprime los números dos veces
- B) Lanza IllegalStateException porque un stream ya consumido no puede reutilizarse
- C) No compila
- D) Imprime los números solo una vez y no hace nada la segunda vez

136. ¿Qué imprime este código?

```
List<String> nombres = List.of("Ana", "Bruno", "Carla");  
String resultado = nombres.stream()  
    .reduce("", (a, b) -> a + b.charAt(0));  
System.out.println(resultado);
```

- A) ABC
- B) AnaBrunoCarla
- C) abc
- D) No compila

137. ¿Cuál es la firma funcional de la interfaz Function<T, R>?

- A) R apply(T t)
- B) void accept(T t)
- C) boolean test(T t)
- D) T get()

138. ¿Qué imprime este código?

```
List<Integer> nums = List.of(3, 1, 4, 1, 5);  
Optional<Integer> max = nums.stream().max(Integer::compareTo);  
System.out.println(max.get());
```

- A) 5
- B) 1
- C) 4
- D) Optional.empty

139. ¿Qué imprime este código?

```
var resultado = Stream.iterate(1, n -> n * 2)  
    .limit(4)  
    .collect(Collectors.toList());  
System.out.println(resultado);
```

- A) [1, 2, 4, 8]
- B) [2, 4, 6, 8]
- C) [1, 2, 3, 4]
- D) No compila

140. ¿Qué imprime este código?

```
Map<Boolean, List<Integer>> particion = Stream.of(1, 2, 3, 4, 5)  
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));  
System.out.println(particion.get(true));
```

- A) [2, 4]
- B) [1, 3, 5]
- C) [1, 2, 3, 4, 5]
- D) No compila

141. ¿Qué diferencia existe entre map() y flatMap() en Stream?

- A) map() transforma cada elemento en otro elemento; flatMap() transforma cada elemento en un stream y luego los 'aplana' en uno solo
- B) flatMap() siempre es más rápido que map()
- C) map() solo funciona con tipos numéricos
- D) No hay ninguna diferencia real

142. ¿Qué imprime este código?

```
IntStream.rangeClosed(1, 5)  
    .forEach(System.out::print);
```

- A) 12345
- B) 1234
- C) 0 1 2 3 4
- D) 12345 (con espacios)

143. ¿Qué imprime este código?

```
BinaryOperator<Integer> suma = (a, b) -> a + b;  
System.out.println(suma.apply(3, 4));
```

- A) 7
- B) 34
- C) No compila
- D) 12

144. ¿Qué imprime este código?

```
List<String> palabras = List.of("uno", "dos", "tres");  
String unido = palabras.stream()  
    .collect(Collectors.joining(" ", "[", "]"));  
System.out.println(unido);
```

- A) [uno, dos, tres]
- B) uno, dos, tres
- C) [uno][dos][tres]
- D) No compila

145. ¿Cuál de las siguientes operaciones de Stream es una operación INTERMEDIA (perezosa, lazy)?

- A) forEach()
- B) collect()
- C) sorted()
- D) count()

146. ¿Qué es cierto sobre peek() en este código?

```
List<Integer> nums = List.of(1, 2, 3);  
long total = nums.stream()  
    .peek(n -> System.out.print("v" + n))  
    .filter(n -> n > 1)  
    .count();
```

- A) Es una operación intermedia usada típicamente para depuración, que no modifica el stream
- B) Es una operación terminal que reemplaza a forEach
- C) Falla en tiempo de compilación
- D) Siempre debe ir después de un filter

147. ¿Qué imprime este código?

```
List<String> nombres = List.of("Ana", "Bruno", "Carla");  
Map<String, Integer> mapa = nombres.stream()  
    .collect(Collectors.toMap(n -> n, String::length));  
System.out.println(mapa.get("Bruno"));
```

- A) 5
- B) 6
- C) 0
- D) No compila

148. ¿Qué imprime este código?

```
Stream.generate(() -> "x").limit(3).forEach(System.out::print);
```

- A) xxx
- B) x
- C) Bucle infinito
- D) No compila

149. ¿Qué imprime este código?

```
List<Integer> nums = List.of(2, 4, 6, 8);  
System.out.println(nums.stream().allMatch(n -> n % 2 == 0));  
System.out.println(nums.stream().anyMatch(n -> n > 6));
```

- A) true true
- B) false false
- C) true false
- D) false true

150. ¿Qué imprime este código?

```
int[] arr = {1, 2, 3};  
Stream<int[]> s1 = Stream.of(arr);  
IntStream s2 = Arrays.stream(arr);  
System.out.println(s1.count() + " " + s2.count());
```

- A) 1 3
- B) 3 3
- C) 3 1
- D) No compila

151. ¿Qué imprime este código?

```
List<String> palabras = List.of("uno", "dos", "tres", "cuatro", "cinco");  
Map<Integer, Long> porLongitud = palabras.stream()  
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));  
System.out.println(porLongitud.get(4));
```

- A) 2
- B) 1
- C) 3
- D) No compila

152. ¿Qué ventaja tiene `orElseGet()` sobre `orElse()` cuando el `Optional` ya contiene un valor?

- A) `orElseGet()` no ejecuta el `Supplier` si el valor ya está presente, evitando cálculos innecesarios; `orElse()` siempre evalúa su argumento
- B) No hay diferencia alguna entre ambos
- C) `orElse()` es siempre más rápido
- D) `orElseGet()` lanza una excepción si el `Optional` tiene valor

153. ¿Qué imprime este código?

```
List<String> lista = new ArrayList<>(List.of("banana", "kiwi", "uva"));
lista.stream()
    .sorted(Comparator.comparing(String::length).reversed())
    .forEach(System.out::print);
```

- A) bananakiwiuva
- B) uvakiwibanana
- C) kiwiuvabanana
- D) No compila

154. ¿Qué imprime este código?

```
String resultado = IntStream.range(1, 4)
    .mapToObj(i -> "n" + i)
    .collect(Collectors.joining(","));
System.out.println(resultado);
```

- A) n1,n2,n3
- B) n0,n1,n2
- C) 1,2,3
- D) No compila

155. ¿Qué imprime este código?

```
List<Integer> nums = List.of(2, 4, 6);
Double promedio = nums.stream().collect(Collectors.averagingInt(n -> n));
System.out.println(promedio);
```

- A) 4.0
- B) 3
- C) 12
- D) No compila

156. ¿Qué imprime este código?

```
Stream<Integer> s1 = Stream.of(1, 2);
Stream<Integer> s2 = Stream.of(3, 4);
Stream.concat(s1, s2).forEach(System.out::print);
```

- A) 1234
- B) 3412
- C) 12 34
- D) No compila

157. ¿Qué imprime este código?

```
record Punto(int x, int y) {}  
List<Punto> puntos = List.of(new Punto(1, 1), new Punto(1, 1), new Punto(2, 2));  
long total = puntos.stream().distinct().count();  
System.out.println(total);
```

- A) 2
- B) 3
- C) 1
- D) No compila

158. ¿Qué imprime este código?

```
Function<Integer, Integer> sumarUno = x -> x + 1;  
Function<Integer, Integer> multiplicarDos = x -> x * 2;  
System.out.println(sumarUno.andThen(multiplicarDos).apply(3));
```

- A) 8
- B) 7
- C) 6
- D) No compila

159. ¿Qué imprime este código?

```
Predicate<Integer> esPar = n -> n % 2 == 0;  
Predicate<Integer> esImpar = esPar.negate();  
System.out.println(esImpar.test(3));
```

- A) true
- B) false
- C) No compila
- D) Error en tiempo de ejecución

160. ¿Qué imprime este código?

```
Optional<Integer> primero = Stream.of(1, 2, 3, 4, 5)  
    .filter(n -> n > 2)  
    .findFirst();  
System.out.println(primerο.get());
```

- A) 3
- B) 1
- C) 5
- D) Optional.empty

7. Concurrency

161. ¿Qué ocurre al ejecutar este código?

```
Runnable tarea = () -> System.out.println("Ejecutando");  
Thread t = new Thread(tarea);  
t.start();  
t.start();
```

- A) Se imprime 'Ejecutando' dos veces
- B) Se lanza `IllegalThreadStateException` en la segunda llamada a `start()`
- C) No compila
- D) El hilo se pausa indefinidamente

162. ¿Qué hace el método `shutdown()` en este contexto?

```
ExecutorService executor = Executors.newFixedThreadPool(2);  
executor.submit(() -> System.out.println("Tarea 1"));  
executor.submit(() -> System.out.println("Tarea 2"));  
executor.shutdown();
```

- A) Detiene inmediatamente todas las tareas en ejecución
- B) Permite que las tareas ya enviadas terminen, pero rechaza nuevas tareas
- C) Reinicia el pool de hilos
- D) Bloquea el hilo principal hasta que termine todo

163. ¿Qué garantiza la palabra clave `'synchronized'` en este método?

```
private int contador = 0;  
  
public synchronized void incrementar() {  
    contador++;  
}
```

- A) Que el método se ejecute más rápido
- B) Que solo un hilo a la vez pueda ejecutar este método sobre la misma instancia, evitando condiciones de carrera
- C) Que el método se ejecute en un hilo separado automáticamente
- D) Que el contador nunca pueda ser negativo

164. ¿Cuál es la diferencia principal entre `Callable<V>` y `Runnable`?

- A) `Callable` puede devolver un valor y lanzar excepciones checked; `Runnable` no devuelve valor
- B) `Runnable` es más moderno que `Callable`
- C) `Callable` no puede ejecutarse en un `ExecutorService`
- D) No hay ninguna diferencia funcional

165. ¿Qué imprime este código?

```
AtomicInteger contador = new AtomicInteger(0);
contador.incrementAndGet();
contador.incrementAndGet();
System.out.println(contador.get());
```

- A) 0
- B) 1
- C) 2
- D) No compila

166. ¿Qué imprime este código?

```
ExecutorService ex = Executors.newSingleThreadExecutor();
Future<Integer> resultado = ex.submit(() -> 5 + 5);
System.out.println(resultado.get());
ex.shutdown();
```

- A) 10
- B) 0
- C) Lanza una excepción checked no manejada obligatoriamente en tiempo de compilación
- D) No compila

167. ¿Qué problema de concurrencia describe mejor un 'deadlock'?

- A) Dos o más hilos quedan bloqueados esperando indefinidamente recursos que el otro posee
- B) Un hilo consume el 100% de CPU sin terminar nunca
- C) Dos hilos leen la misma variable al mismo tiempo
- D) Un hilo termina antes de lo esperado

168. ¿Qué ocurre al ejecutar este código?

```
CopyOnWriteArrayList<String> lista = new CopyOnWriteArrayList<>();
lista.add("a");
for (String s : lista) {
    lista.add("b");
}
System.out.println(lista.size());
```

- A) Lanza ConcurrentModificationException
- B) Se ejecuta sin error, imprimiendo 2 (el bucle solo itera sobre el snapshot original)
- C) Bucle infinito
- D) No compila

169. ¿Cuál es el propósito principal de un objeto CountdownLatch?

- A) Hacer que uno o más hilos esperen hasta que un conjunto de operaciones realizadas en otros hilos se complete
- B) Contar cuántos hilos han terminado con excepción
- C) Reemplazar por completo a ExecutorService
- D) Ordenar alfabéticamente los hilos en ejecución

170. ¿Qué método de la clase Thread NO se recomienda usar porque puede dejar objetos en un estado inconsistente?

- A) Thread.sleep()
- B) Thread.stop()
- C) Thread.join()
- D) Thread.interrupt()

171. ¿Qué garantiza la palabra clave 'volatile' aplicada a un campo?

- A) Que los cambios hechos por un hilo sean visibles inmediatamente para otros hilos, evitando cachés locales desactualizadas
- B) Que el campo sea thread-safe para operaciones compuestas como incrementos
- C) Que el campo no pueda modificarse nunca
- D) Que el campo se serialice automáticamente

172. ¿Por qué se recomienda llamar a unlock() dentro de un bloque finally?

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    // sección crítica
} finally {
    lock.unlock();
}
```

- A) Para garantizar que el lock se libere incluso si ocurre una excepción en la sección crítica
- B) Porque lock() lo requiere obligatoriamente por sintaxis
- C) Para mejorar el rendimiento
- D) No es necesario, Java libera locks automáticamente

173. ¿Qué se requiere para invocar wait() sobre un objeto?

```
synchronized(objeto) {
    objeto.wait();
}
```

- A) Estar dentro de un bloque synchronized sobre ese mismo objeto, o se lanza IllegalMonitorStateException
- B) Nada especial, puede llamarse en cualquier contexto
- C) Que el objeto implemente Runnable
- D) Que el hilo actual sea el hilo principal

174. ¿Qué garantiza establecer una prioridad alta con setPriority() en un Thread?

- A) Nada garantizado de forma determinista: es solo una sugerencia al planificador del sistema operativo
- B) Que ese hilo siempre se ejecutará antes que los demás
- C) Que ese hilo no podrá ser interrumpido
- D) Que el hilo se convierte automáticamente en daemon

175. ¿Qué propósito cumple ThreadLocal en este código?

```
ThreadLocal<Integer> contador = ThreadLocal.withInitial(() -> 0);  
contador.set(contador.get() + 1);  
System.out.println(contador.get());
```

- A) Mantener una copia independiente de la variable para cada hilo que la use
- B) Sincronizar el acceso entre múltiples hilos
- C) Hacer que la variable sea inmutable
- D) Serializar el valor entre hilos

176. ¿Qué comportamiento tiene invokeAll()?

```
List<Callable<Integer>> tareas = List.of(() -> 1, () -> 2, () -> 3);  
List<Future<Integer>> resultados = executor.invokeAll(tareas);
```

- A) Ejecuta todas las tareas y bloquea hasta que TODAS terminen, devolviendo sus Future
- B) Ejecuta solo la primera tarea que termine
- C) No bloquea el hilo llamador
- D) Requiere que las tareas sean Runnable, no Callable

177. ¿Cuál es una ventaja de implementar Runnable en lugar de extender Thread directamente?

- A) Permite que la clase siga pudiendo extender otra clase, ya que Java no permite herencia múltiple
- B) Runnable es más rápido en tiempo de ejecución
- C) Solo Runnable puede usarse con ExecutorService
- D) Thread no puede iniciar más de un hilo

178. ¿Qué imprime este código?

```
CompletableFuture<Integer> cf = CompletableFuture.supplyAsync(() -> 5)  
    .thenApply(x -> x * 2);  
System.out.println(cf.get());
```

- A) 10
- B) 5
- C) No compila
- D) 0

179. ¿Sobre qué objeto se aplica el lock cuando 'synchronized' se usa en un método estático?

```
class Contador {  
    static int valor = 0;  
    static synchronized void incrementar() { valor++; }  
}
```

- A) Sobre el objeto Class asociado a la clase Contador, no sobre una instancia particular
- B) Sobre la primera instancia creada de Contador
- C) No se aplica ningún lock real
- D) Sobre el hilo actual

180. ¿Qué problema tiene este código?

```
public void metodo() {  
    Thread.sleep(1000);  
}
```

- A) No compila porque Thread.sleep() lanza InterruptedException (checked) que debe manejarse o declararse
- B) No compila porque sleep() requiere un objeto Thread explícito
- C) Compila y funciona sin problema
- D) sleep() no existe en la clase Thread

181. ¿Qué hace scheduleAtFixedRate() en este contexto?

```
ScheduledExecutorService ses = Executors.newScheduledThreadPool(1);  
ses.scheduleAtFixedRate(() -> System.out.println("tick"), 0, 1, TimeUnit.SECONDS);
```

- A) Ejecuta la tarea repetidamente cada cierto intervalo de tiempo fijo, comenzando de inmediato
- B) Ejecuta la tarea una sola vez tras el retraso indicado
- C) Cancela automáticamente después de la primera ejecución
- D) Solo funciona con Callable

182. ¿Qué ventaja ofrece ConcurrentHashMap sobre un HashMap sincronizado manualmente (Collections.synchronizedMap)?

- A) Permite mayor concurrencia al usar particionamiento interno con bloqueos más finos, en lugar de bloquear todo el mapa en cada operación
- B) ConcurrentHashMap no permite valores null y HashMap sí, siendo esa la única diferencia relevante
- C) Es idéntico en rendimiento pero más simple de usar
- D) Un HashMap sincronizado siempre es más rápido

183. ¿Qué implica marcar un hilo como 'daemon'?

```
Thread hilo = new Thread(() -> tareaLarga());  
hilo.setDaemon(true);  
hilo.start();
```

- A) La JVM puede terminar el proceso aunque este hilo siga en ejecución, ya que no impide la salida del programa
- B) El hilo tendrá prioridad máxima
- C) El hilo no puede ser interrumpido
- D) El hilo se ejecuta en un proceso separado

184. ¿Qué controla un Semaphore inicializado con 2 permisos?

```
Semaphore sem = new Semaphore(2);
sem.acquire();
try {
    // recurso limitado
} finally {
    sem.release();
}
```

- A) Que como máximo 2 hilos accedan simultáneamente a la sección protegida
- B) Que solo 2 hilos puedan crearse en total en la aplicación
- C) Que la sección se ejecute exactamente 2 veces
- D) Que el recurso se libere automáticamente tras 2 segundos

185. Si múltiples hilos llaman a incrementar() concurrentemente sin sincronización, ¿qué puede ocurrir?

```
class Contador {
    int valor = 0;
    void incrementar() { valor++; } // sin synchronized
}
```

- A) El valor final puede ser menor al esperado debido a una condición de carrera (race condition), ya que valor++ no es atómico
- B) El código no compila
- C) Siempre se obtiene el resultado correcto porque int es primitivo
- D) Se lanza ConcurrentModificationException

186. ¿Cuál de los siguientes NO es un estado válido de Thread.State en Java?

- A) NEW
- B) RUNNABLE
- C) PAUSED
- D) TERMINATED

187. ¿Qué efecto tiene la llamada a t.join()?

```
Thread t = new Thread(() -> tareaLarga());
t.start();
t.join();
System.out.println("terminado");
```

- A) Bloquea el hilo actual hasta que el hilo 't' finalice su ejecución
- B) Termina inmediatamente el hilo 't'
- C) Inicia el hilo 't' nuevamente
- D) No tiene efecto si 't' ya fue iniciado

8. Java I/O y NIO.2

188. En un sistema tipo Unix, ¿qué imprime aproximadamente este código?

```
Path p = Path.of("datos", "archivo.txt");
System.out.println(p);
```

- A) datos/archivo.txt
- B) datos\\archivo.txt
- C) datosarchivo.txt
- D) No compila

189. ¿Cuál es la diferencia principal entre Files.readAllLines() y Files.lines()?

- A) readAllLines() carga todo el archivo en memoria como una List<String>; lines() devuelve un Stream perezoso, más adecuado para archivos grandes
- B) lines() solo puede leer archivos binarios
- C) readAllLines() no puede lanzar IOException
- D) No hay ninguna diferencia

190. ¿Qué característica de diseño demuestra este código?

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
}
```

- A) Un patrón decorador: BufferedReader envuelve a FileReader para añadir buffering
- B) Serialización de objetos
- C) Uso de streams funcionales (java.util.stream)
- D) Programación reactiva

191. Para que una clase pueda ser escrita con ObjectOutputStream (serialización), ¿qué interfaz debe implementar?

- A) Externalizable únicamente
- B) Serializable
- C) Cloneable
- D) AutoCloseable

192. ¿Qué hace StandardCopyOption.REPLACE_EXISTING en esta llamada?

```
Path origen = Path.of("a.txt");
Path destino = Path.of("b.txt");
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
```

- A) Permite sobrescribir el archivo destino si ya existe
- B) Elimina el archivo origen tras copiarlo
- C) Crea automáticamente las carpetas necesarias
- D) Comprime el archivo copiado

193. ¿Qué método de la interfaz Path se usa para obtener la ruta absoluta a partir de una ruta relativa?

- A) toAbsolutePath()
- B) resolve()
- C) normalize()
- D) relativize()

194. ¿Qué hace Files.walk() en este contexto?

```
try (Stream<Path> paths = Files.walk(Path.of("."))) {  
    long total = paths.filter(Files::isRegularFile).count();  
    System.out.println(total);  
}
```

- A) Recorre recursivamente el árbol de directorios a partir de la ruta dada, devolviendo un Stream<Path>
- B) Solo lista los archivos del directorio actual, sin recursividad
- C) Elimina todos los archivos del directorio
- D) Crea un nuevo directorio

195. ¿Qué anotación se usa en un campo para que NO sea incluido en la serialización de un objeto?

- A) @NotSerializable
- B) transient (modificador, no anotación)
- C) @Ignore
- D) static (modificador)

196. Si el archivo no existe, ¿qué imprime este código?

```
Path p = Path.of("no_existe.txt");  
System.out.println(Files.exists(p));
```

- A) false
- B) true
- C) Lanza excepción
- D) No compila

197. Si los directorios intermedios 'a' y 'a/b' no existen, ¿qué hace esta línea?

```
Path dir = Path.of("a/b/c");  
Files.createDirectories(dir);
```

- A) Los crea automáticamente junto con 'c'
- B) Lanza NoSuchFileException porque faltan directorios intermedios
- C) Solo crea 'c' si 'a/b' ya existe
- D) No compila

198. Si el archivo 'salida.txt' no existe, ¿qué hace esta línea?

```
Files.write(Path.of("salida.txt"), "Hola".getBytes());
```

- A) Crea el archivo y escribe el contenido indicado
- B) Lanza NoSuchFileException porque el archivo no existe
- C) No compila
- D) Falla silenciosamente sin escribir nada

199. ¿Qué hace el segundo argumento 'true' en este constructor de FileWriter?

```
FileWriter fw = new FileWriter("log.txt", true);  
fw.write("nueva línea");  
fw.close();
```

- A) Habilita el modo append, agregando contenido al final del archivo sin sobrescribirlo
- B) Hace que el archivo se cree como de solo lectura
- C) Activa el buffering automático
- D) Convierte el escritor en thread-safe

200. ¿Qué ventaja aporta PrintWriter frente a escribir directamente con FileWriter?

```
PrintWriter pw = new PrintWriter(new FileWriter("datos.txt"));  
pw.println("Hola");  
pw.close();
```

- A) Ofrece métodos convenientes como println(), print() y printf() para distintos tipos de datos
- B) Es la única forma de escribir texto en un archivo
- C) Cifra automáticamente el contenido
- D) Convierte el archivo en inmutable

201. ¿Qué ocurre si el archivo no existe al llamar a Files.delete()?

```
Files.delete(Path.of("no_existe.txt"));
```

- A) Lanza NoSuchFileException
- B) No hace nada silenciosamente
- C) Crea el archivo antes de eliminarlo
- D) No compila

202. ¿Qué imprime este código?

```
Path base = Path.of("/datos/proyectos");  
Path destino = Path.of("/datos/proyectos/java/App.java");  
System.out.println(base.relativize(destino));
```

- A) java/App.java
- B) proyectos/java/App.java
- C) ../App.java
- D) No compila

203. ¿Qué imprime este código?

```
Path base = Path.of("/datos");  
Path resultado = base.resolve("proyectos/app.txt");  
System.out.println(resultado);
```

- A) /datos/proyectos/app.txt
- B) proyectos/app.txt
- C) /datos
- D) No compila

204. ¿Cuál es el propósito principal de declarar un campo 'serialVersionUID' en una clase Serializable?

- A) Identificar de forma consistente la versión de la clase durante la deserialización, evitando InvalidClassException si la clase cambia
- B) Es obligatorio para que la clase compile
- C) Cifra el contenido serializado
- D) Aumenta el rendimiento de la serialización

205. ¿Por qué es necesario el cast explícito a (Persona) en esta línea?

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("datos.ser"));  
Persona p = (Persona) ois.readObject();
```

- A) Porque readObject() devuelve Object de forma genérica, sin importar el tipo real serializado
- B) Porque Persona no implementa Serializable
- C) Porque ObjectInputStream no puede leer objetos personalizados
- D) No es necesario, es opcional

206. ¿Para qué se utiliza típicamente la clase java.io.Console?

- A) Para leer entradas interactivas del usuario desde la terminal, incluyendo contraseñas ocultas con readPassword()
- B) Para escribir archivos binarios
- C) Para gestionar hilos de ejecución
- D) Para serializar objetos

207. ¿En qué orden se cierran 'a' y 'b' al finalizar el bloque?

```
try (var a = new RecursoA(); var b = new RecursoB()) {  
    // uso  
}
```

- A) Primero b, luego a (orden inverso a la declaración)
- B) Primero a, luego b
- C) En orden aleatorio
- D) Solo se cierra el último declarado

208. ¿Qué ventaja tiene usar `bw.newLine()` en lugar de escribir `"\n"` directamente?

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {  
    bw.write("linea1");  
    bw.newLine();  
    bw.write("linea2");  
}
```

- A) Usa el separador de línea correcto según el sistema operativo (`line.separator`)
- B) Es más rápido en todos los casos
- C) Cifra automáticamente el salto de línea
- D) No hay ninguna diferencia real

209. ¿Qué devuelve `Files.size()`?

```
long tamano = Files.size(Path.of("datos.txt"));
```

- A) El tamaño del archivo en bytes
- B) El número de líneas del archivo
- C) El número de caracteres del archivo
- D) La fecha de creación del archivo

210. ¿Qué hace `Files.newDirectoryStream()` en este ejemplo?

```
try (DirectoryStream<Path> stream = Files.newDirectoryStream(Path.of("."))) {  
    for (Path p : stream) {  
        System.out.println(p.getFileName());  
    }  
}
```

- A) Lista los archivos y directorios directamente contenidos en la ruta dada (sin recursividad)
- B) Recorre recursivamente todos los subdirectorios
- C) Elimina todo el contenido del directorio
- D) Solo funciona con archivos `.txt`

211. ¿Qué característica distingue a `RandomAccessFile` de otras clases de E/S como `FileInputStream`?

- A) Permite leer y escribir en cualquier posición del archivo usando `seek()`, sin tener que procesar secuencialmente
- B) Solo permite lectura, nunca escritura
- C) Es la única clase que implementa `Serializable`
- D) No requiere cerrarse nunca

212. ¿Qué función cumple `InputStreamReader` en este código?

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
String linea = br.readLine();
```

- A) Actúa como puente (bridge) entre un flujo de bytes (`InputStream`) y un flujo de caracteres (`Reader`), aplicando una codificación de texto
- B) Comprime los datos leídos
- C) Cifra la entrada del usuario
- D) Convierte automáticamente el texto a mayúsculas

213. ¿Qué hace esta línea?

```
Files.move(Path.of("origen.txt"), Path.of("destino.txt"), StandardCopyOption.REPLACE_EXISTING);
```

- A) Mueve (renombra) el archivo origen.txt a destino.txt, sobrescribiendo si ya existe
- B) Copia el archivo sin eliminar el original
- C) Solo funciona dentro del mismo directorio
- D) No compila

214. ¿Qué ocurre al intentar serializar una instancia de Sesion?

```
class Conexion {} // no implementa Serializable
class Sesion implements Serializable {
    Conexion conexion = new Conexion();
}
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("s.ser"));
oos.writeObject(new Sesion());
```

- A) Se lanza NotSerializableException en tiempo de ejecución porque Conexion no implementa Serializable
- B) Se serializa correctamente ignorando el campo conexion
- C) No compila
- D) Se lanza IOException genérica sin más detalles

9. JDBC (Java Database Connectivity)

215. ¿Por qué se prefiere PreparedStatement sobre Statement en este ejemplo?

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement("SELECT * FROM usuarios WHERE id = ?")) {
    ps.setInt(1, 5);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

- A) Porque previene inyección SQL y permite reutilizar el plan de ejecución con parámetros
- B) Porque no requiere manejo de excepciones
- C) Porque es la única forma de ejecutar SELECT
- D) Porque no necesita cerrarse nunca

216. ¿Qué método de ResultSet se usa para avanzar al siguiente registro y verificar si existe?

- A) next()
- B) hasNext()
- C) advance()
- D) getRow()

217. ¿Qué patrón demuestra este código?

```
Connection conn = DriverManager.getConnection(url, user, pass);
conn.setAutoCommit(false);
try {
    // varias operaciones
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}
```

- A) Manejo manual de transacciones: desactivar auto-commit, confirmar o revertir cambios explícitamente
- B) Pool de conexiones
- C) Serialización JDBC
- D) Batch processing obligatorio

218. ¿Cuál es el propósito de la interfaz DriverManager en JDBC?

- A) Gestionar un registro de drivers JDBC y establecer conexiones a bases de datos según la URL proporcionada
- B) Ejecutar directamente sentencias SQL sin necesidad de Connection
- C) Serializar resultados a JSON automáticamente
- D) Sustituir completamente a los drivers específicos de cada base de datos

219. ¿Qué ventaja principal ofrece el uso de addBatch() y executeBatch()?

```
String sql = "INSERT INTO log(mensaje) VALUES (?)";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "evento1");
    ps.addBatch();
    ps.setString(1, "evento2");
    ps.addBatch();
    int[] resultados = ps.executeBatch();
}
```

- A) Ejecutar múltiples sentencias SQL en un solo viaje de red, mejorando el rendimiento
- B) Evitar el uso de PreparedStatement
- C) Garantizar que las inserciones sean atómicas automáticamente sin transacciones
- D) Permitir ejecutar sentencias DDL únicamente

220. ¿Qué diferencia hay entre Statement y CallableStatement?

- A) CallableStatement se usa específicamente para invocar procedimientos almacenados
- B) Statement no permite ejecutar SELECT
- C) CallableStatement no puede recibir parámetros
- D) No existe ninguna diferencia real

221. ¿Cuál es el formato típico de una URL de conexión JDBC para MySQL?

- A) jdbc:mysql://host:puerto/basededatos
- B) mysql://host/basededatos
- C) host:puerto/basededatos
- D) db:mysql://host/basededatos

222. ¿Qué ventaja aporta usar try-with-resources con Connection?

```
try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    // usar conexión
}
```

- A) Garantiza que conn.close() se invoque automáticamente, liberando recursos incluso si ocurre una excepción
- B) Hace que las consultas se ejecuten más rápido
- C) Evita tener que usar PreparedStatement
- D) Permite omitir el manejo de SQLException

223. ¿Qué es cierto sobre el acceso a columnas de un ResultSet?

```
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    int id = rs.getInt("id");
    String nombre = rs.getString(2);
}
```

- A) Se puede acceder por nombre de columna o por índice (basado en 1, no en 0)
- B) Solo se puede acceder por nombre de columna
- C) Los índices de columna comienzan en 0
- D) No se puede mezclar acceso por nombre e índice

224. ¿SQLException es una excepción checked o unchecked?

- A) Checked, extiende directamente de Exception
- B) Unchecked, extiende de RuntimeException
- C) Es un Error, no una Exception
- D) Depende del driver JDBC usado

225. ¿Qué vulnerabilidad de seguridad presenta este código?

```
String sql = "SELECT * FROM usuarios WHERE nombre = '" + entradaUsuario + "'";  
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery(sql);
```

- A) Inyección SQL, ya que 'entradaUsuario' se concatena directamente sin sanitizar ni usar parámetros
- B) Ninguna, es una práctica segura y recomendada
- C) Fuga de memoria
- D) Bloqueo de la tabla usuarios

226. Por defecto, ¿qué tipo de cursor tiene un ResultSet obtenido con executeQuery()?

- A) TYPE_FORWARD_ONLY: solo permite avanzar con next(), no retroceder
- B) TYPE_SCROLL_SENSITIVE, permite navegación bidireccional
- C) Aleatorio según el driver
- D) Siempre de solo lectura y bidireccional

227. ¿Qué ocurre si no se cierra una Connection JDBC explícitamente tras usarla?

- A) Se puede agotar el pool de conexiones disponibles, causando fugas de recursos en la aplicación
- B) No pasa nada, Java la cierra automáticamente siempre
- C) Se cierra automáticamente al final del método actual
- D) La base de datos la cierra tras 1 segundo

228. ¿Qué representa cada elemento del array devuelto por executeBatch()?

```
int[] resultados = ps.executeBatch();
```

- A) El número de filas afectadas por cada sentencia del lote (o un código especial si no se puede determinar)
- B) El tiempo en milisegundos que tardó cada sentencia
- C) El identificador generado por cada inserción
- D) Siempre son ceros

229. ¿Para qué se usa Statement.RETURN_GENERATED_KEYS junto con getGeneratedKeys()?

```
PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);  
ps.executeUpdate();  
ResultSet keys = ps.getGeneratedKeys();
```

- A) Para obtener valores generados automáticamente por la base de datos, como claves primarias autoincrementales, tras una inserción
- B) Para revertir la transacción si falla
- C) Para cifrar las claves de la tabla
- D) Para validar el esquema de la tabla

230. En versiones modernas de JDBC (4.0+), ¿es obligatorio llamar a `Class.forName("driver")` antes de conectar?

- A) No, los drivers JDBC 4.0+ se registran automáticamente mediante el mecanismo `ServiceLoader`
- B) Sí, siempre es obligatorio en cualquier versión
- C) Solo es necesario para bases de datos Oracle
- D) Solo es necesario si se usa un pool de conexiones

231. ¿Qué controla el nivel de aislamiento de una transacción?

```
conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
```

- A) Qué tan visibles son los cambios no confirmados de otras transacciones concurrentes para la transacción actual
- B) La velocidad de ejecución de las consultas
- C) El número máximo de conexiones simultáneas
- D) El tipo de driver JDBC utilizado

232. ¿Qué información provee `ResultSetMetaData`?

```
ResultSetMetaData meta = rs.getMetaData();  
int columnas = meta.getColumnCount();
```

- A) Metadatos sobre la estructura del resultado, como número de columnas, sus nombres y tipos
- B) Los datos reales de las filas del `ResultSet`
- C) El plan de ejecución SQL
- D) La configuración del driver JDBC

233. ¿Qué permite el tipo `TYPE_SCROLL_INSENSITIVE` en un `ResultSet`?

```
Statement st = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);  
ResultSet rs = st.executeQuery(sql);  
rs.last();  
rs.first();
```

- A) Navegar en ambas direcciones (adelante y atrás) e ir a posiciones específicas como `first()/last()`
- B) Solo avanzar con `next()`, igual que el tipo por defecto
- C) Modificar directamente los datos de la base de datos
- D) Ejecutar consultas más rápido siempre

234. ¿Qué ventaja ofrece usar una interfaz `DataSource` en lugar de `DriverManager` directamente en aplicaciones empresariales?

- A) Permite configurar pooling de conexiones y desacoplar la configuración de la conexión del código, apto para servidores de aplicaciones
- B) `DataSource` solo funciona con `SQLite`
- C) `DriverManager` no puede usarse nunca en producción
- D) No hay ninguna diferencia real

235. ¿Para qué sirve rs.wasNull() después de llamar a un getter como getInt()?

```
int valor = rs.getInt("cantidad");  
if (rs.wasNull()) {  
    System.out.println("el valor era NULL en la base de datos");  
}
```

- A) Para verificar si el último valor leído era NULL en la base de datos (ya que getInt() devuelve 0 tanto para NULL como para 0 real)
- B) Para verificar si la conexión sigue activa
- C) Para cerrar el ResultSet
- D) Para revertir la última operación

236. ¿Qué representa el valor devuelto por executeUpdate() en una sentencia UPDATE o DELETE?

```
int filas = ps.executeUpdate();
```

- A) El número de filas afectadas por la operación
- B) El ID de la última fila insertada
- C) Un booleano indicando éxito o fracaso
- D) El tiempo de ejecución en milisegundos

237. ¿Qué permite un Savepoint dentro de una transacción?

```
Savepoint sp = conn.setSavepoint();  
// operaciones riesgosas  
conn.rollback(sp);
```

- A) Revertir parcialmente una transacción hasta un punto intermedio, sin deshacer todo lo confirmado previamente
- B) Confirmar (commit) parcialmente una transacción
- C) Cambiar el nivel de aislamiento en tiempo real
- D) Cerrar la conexión de forma segura

238. ¿Para qué se usa PreparedStatement.setNull()?

```
ps.setNull(2, Types.INTEGER);
```

- A) Para asignar explícitamente un valor NULL de SQL a un parámetro, indicando el tipo SQL correspondiente
- B) Para eliminar un parámetro previamente asignado
- C) Para cerrar el PreparedStatement
- D) Para validar que el parámetro no sea NULL

239. ¿Cuál es el propósito principal de un pool de conexiones (connection pool) en una aplicación con JDBC?

- A) Reutilizar un conjunto de conexiones ya establecidas para evitar el costo de abrir y cerrar conexiones repetidamente
- B) Cifrar automáticamente todas las consultas SQL
- C) Aumentar el número máximo de tablas permitidas
- D) Sustituir completamente la necesidad de cerrar conexiones

240. ¿Qué hacen absolute(3) y relative(-1) sobre un ResultSet desplazable?

```
rs.absolute(3);  
rs.relative(-1);
```

- A) absolute(3) posiciona el cursor en la fila 3; relative(-1) lo mueve una posición hacia atrás desde la posición actual
- B) Ambos métodos solo funcionan hacia adelante
- C) absolute() lanza excepción siempre
- D) relative() reinicia el cursor al principio

241. ¿Por qué es una buena práctica declarar tanto Statement como ResultSet en el mismo try-with-resources?

```
try (Statement st = conn.createStatement();  
    ResultSet rs = st.executeQuery("SELECT * FROM productos")) {  
    while (rs.next()) {  
        System.out.println(rs.getString("nombre"));  
    }  
}
```

- A) Para asegurar que ambos recursos se cierren automáticamente en el orden correcto, evitando fugas de recursos de la base de datos
- B) Porque ResultSet no puede cerrarse manualmente
- C) Porque es obligatorio por sintaxis del lenguaje
- D) Porque mejora el rendimiento de la consulta SQL

10. Código seguro (secure coding)

242. ¿Qué principio de código seguro ejemplifica mejor esta clase?

```
public final class Punto {  
    private final int x;  
    private final int y;  
    public Punto(int x, int y) { this.x = x; this.y = y; }  
    public int getX() { return x; }  
    public int getY() { return y; }  
}
```

- A) Inmutabilidad: campos privados y finales, clase final, sin setters
- B) Uso de reflexión
- C) Serialización personalizada
- D) Encapsulación de excepciones

243. ¿Por qué se recomienda usar tipos específicos (como List<String>) en lugar de tipos crudos (raw types) como List?

- A) Los genéricos proporcionan seguridad de tipos en tiempo de compilación, evitando ClassCastException en tiempo de ejecución
- B) Los raw types son más rápidos siempre
- C) Los raw types no pueden usarse con colecciones
- D) No hay ninguna diferencia de seguridad

244. ¿Qué práctica de código seguro se demuestra en este método?

```
public void procesar(String entrada) {  
    if (entrada == null || entrada.isBlank()) {  
        throw new IllegalArgumentException("Entrada inválida");  
    }  
    // procesar entrada validada  
}
```

- A) Validación de entradas (input validation) antes de procesarlas
- B) Ofuscación de código
- C) Cifrado de datos
- D) Uso de excepciones checked obligatorias

245. ¿Cuál es una buena práctica al declarar campos de una clase para minimizar la superficie de ataque?

- A) Declarar los campos como private y exponerlos solo mediante métodos controlados (getters, sin setters si no es necesario)
- B) Declarar todos los campos como public para facilitar el acceso
- C) Usar siempre campos static para todo
- D) Evitar el uso de constructores

246. ¿Por qué preferir try-with-resources en lugar de cerrar recursos manualmente en un bloque finally?

- A) Reduce el riesgo de fugas de recursos (resource leaks) al garantizar el cierre automático, incluso si ocurre una excepción
- B) Es la única forma de cerrar un archivo en Java
- C) Hace que el código se ejecute en paralelo
- D) Elimina la necesidad de manejar excepciones

247. ¿Qué riesgo de seguridad se asocia comúnmente con hacer públicos campos mutables de una clase que representa datos sensibles?

- A) Permite que código externo modifique el estado interno sin validación, rompiendo invariantes de la clase
- B) Ningún riesgo real, solo es una cuestión de estilo
- C) Hace que la clase no pueda compilar
- D) Aumenta el uso de memoria significativamente

248. ¿Por qué se realizan copias defensivas de 'fecha' tanto en el constructor como en el getter?

```
public final class Evento {  
    private final Date fecha;  
    public Evento(Date fecha) {  
        this.fecha = new Date(fecha.getTime()); // copia defensiva  
    }  
    public Date getFecha() {  
        return new Date(fecha.getTime()); // copia defensiva  
    }  
}
```

- A) Para evitar que código externo modifique el estado interno del objeto a través de una referencia mutable compartida (Date es mutable)
- B) Para mejorar el rendimiento de la clase
- C) Para permitir la serialización del objeto
- D) Porque Date no puede asignarse directamente a un campo final

249. ¿Qué logra usar List.copyOf() en el constructor?

```
public final class Grupo {  
    private final List<String> miembros;  
    public Grupo(List<String> miembros) {  
        this.miembros = List.copyOf(miembros);  
    }  
    public List<String> getMiembros() { return miembros; }  
}
```

- A) Crear una copia inmutable interna, protegiendo la clase de modificaciones externas posteriores a la lista original pasada al constructor
- B) Ordenar automáticamente la lista
- C) Eliminar duplicados únicamente
- D) Permitir valores null en la lista

250. ¿Qué vulnerabilidad de encapsulación tiene este getter?

```
public class Inventario {  
    private int[] cantidades = {10, 20, 30};  
    public int[] getCantidades() { return cantidades; }  
}
```

- A) Expone una referencia directa al array interno, permitiendo que código externo lo modifique sin control
- B) Ninguna, los arrays son inmutables en Java
- C) El método no compila
- D) Genera una fuga de memoria automáticamente

251. ¿Qué principio de código seguro se aplica al declarar métodos y campos con el modificador de acceso MÁS restrictivo posible que aún permita el funcionamiento correcto?

- A) Principio de mínimo privilegio (least privilege), reduciendo la superficie de ataque de la clase
- B) Principio de sustitución de Liskov
- C) Principio de responsabilidad única
- D) Principio abierto/cerrado

252. ¿Por qué se considera una mala práctica llamar a System.exit() dentro de código de una librería reusable?

- A) Termina abruptamente toda la JVM, afectando a la aplicación completa que use la librería, en lugar de manejar el error localmente
- B) Porque System.exit() no existe en Java
- C) Porque siempre lanza una excepción checked
- D) Porque solo puede llamarse una vez por programa

253. ¿Por qué se recomienda validar el estado del objeto dentro de un método readObject() personalizado?

```
private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {  
    in.defaultReadObject();  
    if (saldo < 0) throw new InvalidObjectException("saldo inválido");  
}
```

- A) Porque un flujo de datos serializado manipulado externamente podría reconstruir un objeto con un estado inválido si no se valida
- B) Porque readObject() nunca se ejecuta automáticamente
- C) Para mejorar el rendimiento de la deserialización
- D) Porque es obligatorio por el compilador

254. ¿Por qué se recomienda usar char[] en lugar de String para almacenar temporalmente contraseñas en memoria?

- A) Porque un char[] puede sobrescribirse y limpiarse explícitamente de memoria, mientras que un String inmutable puede permanecer en memoria (posiblemente en el pool) hasta que el recolector de basura actúe
- B) Porque String no puede contener caracteres especiales
- C) Porque char[] es más rápido de comparar
- D) Porque String no permite más de 8 caracteres

255. ¿Qué problema surge al mutar 'id' después de usar el objeto como clave de un HashMap?

```
class Clave {
    int id;
    Clave(int id) { this.id = id; }
    public boolean equals(Object o) { return o instanceof Clave c && c.id == id; }
    public int hashCode() { return id; }
}
Map<Clave, String> mapa = new HashMap<>();
Clave k = new Clave(1);
mapa.put(k, "valor");
k.id = 2;
System.out.println(mapa.get(k));
```

- A) El objeto puede volverse 'inencontrable' en el mapa porque su hashCode cambió y ya no coincide con el bucket original donde fue almacenado
- B) No hay ningún problema, siempre se encuentra correctamente
- C) El programa lanza una excepción de compilación
- D) El HashMap elimina automáticamente la entrada

256. ¿Qué beneficio de seguridad ofrece declarar una clase sensible (por ejemplo, de validación de permisos) como 'final'?

- A) Evita que código malicioso extienda la clase y sobrescriba métodos para alterar su comportamiento esperado
- B) Hace que la clase se ejecute más rápido siempre
- C) Permite que la clase implemente más interfaces
- D) Es un requisito obligatorio para todas las clases públicas

257. ¿Qué práctica de código seguro se ilustra en este método?

```
public void setCorreo(String correo) {
    if (!correo.matches("^[\\w.-]+@[\\w.-]+\\.[a-z]{2,}$")) {
        throw new IllegalArgumentException("correo inválido");
    }
    this.correo = correo;
}
```

- A) Validación de formato de entrada mediante una expresión regular antes de aceptar el dato
- B) Cifrado del correo electrónico
- C) Serialización segura
- D) Uso de reflexión para validar tipos

258. ¿Qué riesgo de seguridad demuestra este uso de reflexión?

```
Field campo = MiClase.class.getDeclaredField("saldo");
campo.setAccessible(true);
campo.set(objeto, -1000);
```

- A) setAccessible(true) puede saltarse la encapsulación normal (private), permitiendo modificar campos que deberían estar protegidos
- B) Ninguno, es una técnica siempre segura y recomendada
- C) El código no compila nunca
- D) Solo funciona con campos públicos

259. ¿Qué problema de seguridad tiene este manejo de excepciones?

```
try {
    conectar(usuario, clave);
} catch (SQLException e) {
    System.out.println("Error de conexión con usuario=" + usuario + " clave=" + clave);
}
```

- A) Expone información sensible (como la clave) en mensajes de log o consola, lo que puede filtrarse a través de registros
- B) Ninguno, es una práctica recomendada para depuración
- C) No compila porque SQLException no puede capturarse así
- D) Hace que la conexión falle siempre

260. ¿Qué ventaja de diseño ofrece este patrón Builder combinado con inmutabilidad en Config?

```
public final class Config {
    private final String host;
    private final int puerto;
    private Config(Builder b) { this.host = b.host; this.puerto = b.puerto; }
    public static class Builder {
        private String host;
        private int puerto;
        public Builder host(String h) { this.host = h; return this; }
        public Builder puerto(int p) { this.puerto = p; return this; }
        public Config build() { return new Config(this); }
    }
}
```

- A) Permite construir objetos complejos paso a paso, garantizando que el objeto final sea inmutable y consistente una vez creado
- B) Permite modificar Config después de construido sin restricciones
- C) Elimina la necesidad de validación
- D) Hace que Config sea serializable automáticamente

261. ¿Qué riesgo de seguridad presenta este campo estático público y mutable?

```
public class Configuracion {  
    public static List<String> servidores = new ArrayList<>();  
}
```

- A) Cualquier clase puede modificar la lista compartida globalmente sin control ni validación, incluyendo desde hilos concurrentes sin sincronización
- B) Ninguno, es una práctica común y segura
- C) El código no compila
- D) Solo se puede acceder desde la misma clase

262. ¿Qué problema de consistencia tiene esta implementación de clone()?

```
class Pedido implements Cloneable {  
    List<String> items = new ArrayList<>();  
    public Pedido clone() throws CloneNotSupportedException {  
        return (Pedido) super.clone();  
    }  
}
```

- A) Es una copia superficial (shallow copy): el clon comparte la MISMA referencia a la lista 'items', por lo que modificar una afecta a la otra
- B) Es una copia profunda segura, sin ningún problema
- C) No compila porque Cloneable requiere un constructor especial
- D) clone() nunca puede lanzar excepciones checked

263. ¿Por qué se recomienda usar java.security.SecureRandom en lugar de java.util.Random para generar valores sensibles (como tokens de sesión)?

- A) SecureRandom usa fuentes de aleatoriedad criptográficamente más fuertes, dificultando que un atacante prediga los valores generados
- B) Random es más lento que SecureRandom en todos los casos
- C) No hay ninguna diferencia real entre ambas clases
- D) SecureRandom no puede generar números enteros

264. ¿Qué riesgo introduce este patrón de 'tragarse' (swallow) la excepción sin registrarla ni manejarla?

```
try {  
    procesarPago(monto);  
} catch (Exception e) {  
    // no hacer nada  
}
```

- A) Los errores pasan desapercibidos, ocultando fallos reales (incluidos posibles problemas de seguridad) y dificultando la depuración
- B) Mejora la seguridad al ocultar detalles internos
- C) Es la forma recomendada de manejar cualquier excepción
- D) Hace que el código sea más eficiente

265. ¿Qué ataque intenta prevenir esta validación con `normalize()` y `startsWith()`?

```
public void leerArchivo(String nombreArchivo) {  
    Path ruta = Path.of("/datos/usuarios", nombreArchivo).normalize();  
    if (!ruta.startsWith("/datos/usuarios")) {  
        throw new SecurityException("ruta no permitida");  
    }  
    // leer archivo  
}
```

- A) Un ataque de 'path traversal' (por ejemplo, usando '../etc/passwd') que intente acceder a archivos fuera del directorio permitido
- B) Inyección SQL
- C) Ataques de denegación de servicio
- D) Desbordamiento de buffer

266. ¿Qué problema de seguridad presenta codificar credenciales directamente en el código fuente (hardcoding)?

```
String password = "Admin123!";  
conn = DriverManager.getConnection(url, "admin", password);
```

- A) Las credenciales quedan expuestas en el código fuente y en el control de versiones, siendo fácilmente descubiertas si el código se filtra
- B) Ninguno, es la forma más eficiente de conectar a la base de datos
- C) Hace que la conexión sea más rápida
- D) Es un requisito de JDBC

267. ¿Por qué declarar variables en el ámbito (scope) más reducido posible se considera una buena práctica de código seguro y mantenible?

- A) Reduce la posibilidad de uso indebido o modificación accidental desde código que no debería tener acceso a esa variable
- B) Hace que el programa consuma menos memoria siempre
- C) Es un requisito del compilador de Java
- D) Permite que la variable sea accesible desde cualquier clase

268. ¿Qué práctica de código seguro se demuestra en este método?

```
public String obtener(String[] arr, int indice) {  
    if (indice < 0 || indice >= arr.length) {  
        throw new IllegalArgumentException("índice fuera de rango");  
    }  
    return arr[indice];  
}
```

- A) Validar los límites de un índice antes de acceder al array, evitando `ArrayIndexOutOfBoundsException` y posibles fallos explotables
- B) Ordenar el array antes de acceder
- C) Cifrar el contenido del array
- D) Convertir el array en una lista inmutable

11. Aplicaciones modulares (JPMS)

269. ¿Qué permite la directiva 'exports' en este archivo module-info.java?

```
module com.miapp.core {  
    requires java.sql;  
    exports com.miapp.core.servicios;  
}
```

- A) Que otros módulos puedan acceder a las clases públicas del paquete indicado
- B) Que el módulo requiera obligatoriamente java.sql en tiempo de compilación solamente
- C) Que se genere automáticamente documentación Javadoc
- D) Que el paquete se cifre en tiempo de ejecución

270. ¿Qué diferencia hay entre 'requires' y 'requires transitive' en module-info.java?

- A) 'requires transitive' propaga la dependencia a los módulos que a su vez dependan del módulo actual
- B) 'requires transitive' es más rápido en tiempo de ejecución
- C) No existe diferencia real entre ambos
- D) 'requires' solo funciona con módulos de la JDK

271. ¿Para qué se usa típicamente la directiva 'opens ... to'?

```
module com.miapp.core {  
    opens com.miapp.core.modelo to com.miapp.persistencia;  
}
```

- A) Para permitir acceso por reflexión profunda (deep reflection) a un paquete específico, solo desde el módulo indicado
- B) Para exportar el paquete públicamente a todos los módulos
- C) Para importar clases de otro módulo
- D) Para cifrar el paquete en tiempo de compilación

272. ¿Qué comando se usa para ejecutar una aplicación modular desde la línea de comandos?

- A) java --module-path mods -m com.miapp/com.miapp.Main
- B) java -jar com.miapp.Main
- C) javac --module com.miapp
- D) java --run com.miapp.Main

273. ¿Qué es el 'módulo no nombrado' (unnamed module) en JPMS?

- A) El módulo implícito al que pertenece todo el código en el classpath tradicional (sin module-info.java)
- B) Un módulo especial que no puede tener dependencias
- C) El módulo raíz de todas las aplicaciones Java
- D) Un módulo que solo existe en tiempo de compilación

274. ¿Qué directiva de module-info.java se usa para declarar que un módulo proporciona una implementación de un servicio (Service Provider Interface)?

- A) provides ... with ...
- B) implements ... as ...
- C) exports ... as ...
- D) uses ... with ...

275. ¿En qué ubicación debe colocarse el archivo module-info.java dentro del código fuente de un módulo?

- A) En la raíz del árbol de código fuente del módulo (no dentro de ningún paquete)
- B) Dentro del paquete principal de la aplicación
- C) En cualquier subdirectorio, no importa cuál
- D) Debe llamarse igual que el módulo

276. ¿Qué característica tiene el 'módulo no nombrado' respecto a la lectura de otros módulos?

- A) Puede leer (acceder a) todos los demás módulos del module path, pero ningún módulo con nombre puede declarar 'requires' sobre él explícitamente
- B) No puede leer ningún otro módulo
- C) Debe declarar explícitamente todos sus requires
- D) Es idéntico al módulo java.base

277. ¿Qué es un 'módulo automático' en JPMS?

- A) Un JAR sin module-info.java colocado en el module path, al que Java asigna automáticamente un nombre derivado del archivo, y que exporta todos sus paquetes
- B) Un módulo generado automáticamente por el compilador para cada clase
- C) Un módulo que se actualiza solo con nuevas versiones
- D) Un tipo de módulo exclusivo de la JDK

278. ¿Cuál es uno de los principales beneficios del 'encapsulamiento fuerte' (strong encapsulation) que introduce JPMS?

- A) Impide el acceso no autorizado a APIs internas de un módulo que no hayan sido explícitamente exportadas, incluso si las clases son públicas
- B) Elimina la necesidad de modificadores de acceso como private
- C) Hace que todo el código sea automáticamente thread-safe
- D) Permite ejecutar código sin la JVM

279. ¿Para qué se utiliza la herramienta jlink?

- A) Para crear una imagen de runtime personalizada y reducida, que incluya solo los módulos necesarios para ejecutar una aplicación específica
- B) Para compilar archivos .java a .class
- C) Para depurar aplicaciones concurrentes
- D) Para generar documentación Javadoc

280. ¿Para qué se utiliza la herramienta jdeps?

- A) Para analizar las dependencias de un archivo .class o .jar, ayudando a identificar qué módulos requiere una aplicación
- B) Para ejecutar pruebas unitarias automáticamente
- C) Para firmar digitalmente un JAR
- D) Para convertir código legado en módulos automáticamente sin análisis

281. ¿Qué significa la palabra clave 'static' en esta directiva 'requires'?

```
module com.miapp {  
    requires static com.terceros.opcional;  
}
```

- A) Que la dependencia es obligatoria solo en tiempo de compilación, pero opcional en tiempo de ejecución
- B) Que el módulo completo es una clase estática
- C) Que la dependencia nunca podrá actualizarse
- D) Que el módulo no puede exportar paquetes

282. ¿Es válido declarar múltiples directivas 'exports' en un mismo module-info.java?

```
module com.miapp {  
    exports com.miapp.api;  
    exports com.miapp.util;  
}
```

- A) Sí, se pueden exportar tantos paquetes como sea necesario mediante declaraciones 'exports' separadas
- B) No, solo se permite un 'exports' por módulo
- C) Sí, pero como máximo dos
- D) No, se debe usar una sola declaración con comas

283. ¿Qué restricción de JPMS prohíbe que el mismo paquete esté definido simultáneamente en dos módulos diferentes del mismo grafo de módulos?

- A) La prohibición de 'paquetes divididos' (split packages), que causa un error al resolver los módulos en tiempo de lanzamiento
- B) No existe tal restricción, es perfectamente válido
- C) Solo aplica a módulos automáticos
- D) Solo aplica si ambos módulos son de la JDK

284. ¿Qué comando se utiliza para listar los módulos disponibles en la JDK actual?

- A) java --list-modules
- B) java --show-modules
- C) javac --modules
- D) java -modules-list

285. ¿Qué logra una 'exportación calificada' (qualified export) como esta?

```
module com.miapp.core {  
    exports com.miapp.core.interno to com.miapp.tests;  
}
```

- A) Exponer el paquete solo al módulo indicado (com.miapp.tests), manteniéndolo encapsulado para todos los demás
- B) Exportar el paquete públicamente a todos los módulos, igual que un exports normal
- C) Prohibir el acceso incluso al módulo indicado
- D) Requiere que com.miapp.tests sea un módulo automático

286. ¿Qué papel cumple la directiva 'uses' junto con ServiceLoader?

```
module com.miapp {  
    uses com.miapp.api.Procesador;  
}  
ServiceLoader<Procesador> loader = ServiceLoader.load(Procesador.class);
```

- A) Declara que el módulo consume implementaciones de un servicio (Procesador), permitiendo que ServiceLoader las descubra en tiempo de ejecución
- B) Declara que el módulo provee una implementación del servicio
- C) Impide el uso de ServiceLoader
- D) Solo funciona con clases abstractas, no interfaces

287. ¿Se puede usar un comodín (wildcard) para exportar todos los subpaquetes de un paquete en module-info.java, como 'exports com.miapp.*'?

- A) No, cada paquete que se desea exportar debe declararse explícitamente y por completo
- B) Sí, es la forma estándar y recomendada
- C) Sí, pero solo para paquetes de la JDK
- D) No, JPMS no permite exportar más de un paquete por módulo

288. Si se coloca 'commons-utils-2.1.jar' (sin module-info.java) en el module path, ¿qué nombre de módulo automático se le asignaría típicamente?

- A) commons.utils (derivado del nombre del archivo, normalizando guiones y quitando la versión)
- B) commons-utils-2.1
- C) El nombre del primer paquete que contenga
- D) Ninguno; los JAR sin module-info no pueden usarse en el module path

289. Si se especifica una versión al empaquetar un módulo (jar --module-version), ¿JPMS impone automáticamente resolución de conflictos de versiones en tiempo de ejecución?

- A) No, JPMS registra la versión como metadato informativo, pero no realiza resolución de conflictos como lo haría un gestor de dependencias
- B) Sí, JPMS elige automáticamente la versión más reciente disponible
- C) Sí, y lanza un error si hay dos versiones del mismo módulo
- D) No, la versión se ignora completamente y ni siquiera se puede consultar

290. Al empaquetar un módulo ejecutable como JAR modular, ¿cómo se puede indicar la clase principal para ejecutar con solo '-m nombreModulo' (sin especificar la clase)?

- A) Usando la opción --main-class al crear el JAR con la herramienta 'jar', o declarándola en el manifiesto del módulo
- B) Debe especificarse siempre con -m modulo/Clase, sin excepción
- C) Solo es posible con módulos automáticos
- D) Usando 'requires main' en module-info.java

291. Gracias a 'requires transitive' en com.miapp.api, ¿qué acceso obtiene automáticamente com.miapp.web sin declararlo explícitamente?

```
module com.miapp.api {  
    requires transitive com.miapp.modelo;  
}  
module com.miapp.web {  
    requires com.miapp.api;  
}
```

- A) Acceso a los paquetes exportados por com.miapp.modelo, de forma transitiva
- B) Ningún acceso adicional
- C) Acceso total sin restricciones a com.miapp.api
- D) Acceso de solo lectura a com.miapp.modelo

292. ¿Qué diferencia hay entre declarar 'open module' y usar 'opens paquete' selectivamente?

```
open module com.miapp.modelo {  
    exports com.miapp.modelo;  
}
```

- A) 'open module' abre TODOS los paquetes del módulo para reflexión profunda automáticamente, sin necesidad de declarar 'opens' para cada uno
- B) 'open module' es idéntico a no declarar module-info.java
- C) 'open module' impide cualquier exportación
- D) 'open module' solo afecta a un paquete a la vez, igual que 'opens'

293. ¿Qué ocurre si un framework intenta acceder por reflexión profunda a un paquete que NO fue abierto ('opens') por el módulo?

```
// módulo sin 'opens' para el paquete del framework  
Field campo = MiClase.class.getDeclaredField("dato");  
campo.setAccessible(true);
```

- A) Se lanza InaccessibleObjectException en tiempo de ejecución
- B) Se ejecuta sin problema siempre
- C) No compila
- D) Se abre automáticamente el paquete de forma silenciosa

294. ¿Qué tipo de error ocurre típicamente al ejecutar una aplicación modular si un módulo requerido no se encuentra en el module path?

- A) Un error de resolución de módulos en tiempo de lanzamiento (por ejemplo, 'module not found'), antes de ejecutar el código
- B) Un error silencioso que se ignora
- C) Una advertencia que no detiene la ejecución
- D) Un NullPointerException en tiempo de ejecución

295. ¿Cuál es uno de los beneficios principales que JPMS aporta a aplicaciones grandes, conocido como 'configuración confiable' (reliable configuration)?

- A) Detectar en tiempo de lanzamiento dependencias faltantes o conflictivas, en lugar de fallar más tarde con errores como ClassNotFoundException en tiempo de ejecución
- B) Eliminar por completo la necesidad de pruebas unitarias
- C) Permitir que cualquier clase acceda a cualquier otra sin restricciones
- D) Aumentar automáticamente el rendimiento de la JVM

Hoja de respuestas rápida

#	Resp.	#	Resp.	#	Resp.	#	Resp.
1	A	2	B	3	A	4	B
5	B	6	B	7	A	8	A
9	B	10	A	11	B	12	A
13	B	14	B	15	C	16	A
17	A	18	C	19	A	20	A
21	B	22	A	23	B	24	A
25	A	26	A	27	B	28	A
29	B	30	B	31	A	32	A
33	B	34	A	35	B	36	B
37	B	38	B	39	B	40	A
41	A	42	B	43	A	44	B
45	A	46	A	47	A	48	B
49	B	50	B	51	A	52	B
53	A	54	A	55	A	56	B
57	D	58	B	59	B	60	B
61	A	62	A	63	B	64	B
65	A	66	B	67	A	68	A
69	A	70	A	71	A	72	A
73	A	74	B	75	A	76	A
77	A	78	A	79	A	80	B
81	A	82	B	83	B	84	C
85	A	86	B	87	B	88	A
89	B	90	A	91	A	92	A
93	A	94	A	95	A	96	A
97	B	98	A	99	A	100	B
101	B	102	A	103	B	104	B
105	A	106	A	107	B	108	B
109	C	110	B	111	B	112	A
113	B	114	A	115	A	116	B
117	A	118	B	119	A	120	B
121	B	122	A	123	A	124	A

125	B	126	B	127	B	128	A
129	A	130	A	131	B	132	A
133	A	134	A	135	B	136	A
137	A	138	A	139	A	140	A
141	A	142	A	143	A	144	A
145	C	146	A	147	A	148	A
149	A	150	A	151	B	152	A
153	A	154	A	155	A	156	A
157	A	158	A	159	A	160	A
161	B	162	B	163	B	164	A
165	C	166	A	167	A	168	B
169	A	170	B	171	A	172	A
173	A	174	A	175	A	176	A
177	A	178	A	179	A	180	A
181	A	182	A	183	A	184	A
185	A	186	C	187	A	188	A
189	A	190	A	191	B	192	A
193	A	194	A	195	B	196	A
197	A	198	A	199	A	200	A
201	A	202	A	203	A	204	A
205	A	206	A	207	A	208	A
209	A	210	A	211	A	212	A
213	A	214	A	215	A	216	A
217	A	218	A	219	A	220	A
221	A	222	A	223	A	224	A
225	A	226	A	227	A	228	A
229	A	230	A	231	A	232	A
233	A	234	A	235	A	236	A
237	A	238	A	239	A	240	A
241	A	242	A	243	A	244	A
245	A	246	A	247	A	248	A
249	A	250	A	251	A	252	A
253	A	254	A	255	A	256	A
257	A	258	A	259	A	260	A

261	A	262	A	263	A	264	A
265	A	266	A	267	A	268	A
269	A	270	A	271	A	272	A
273	A	274	A	275	A	276	A
277	A	278	A	279	A	280	A
281	A	282	A	283	A	284	A
285	A	286	A	287	A	288	A
289	A	290	A	291	A	292	A
293	A	294	A	295	A		

Respuestas y explicaciones detalladas

1. Bloques básicos de Java (variables, tipos, texto)

1. Respuesta correcta: A) `var x = 10;`

'var' requiere un inicializador para poder inferir el tipo. 'var x;' es inválido porque no hay valor para inferir. 'var x = null;' también es inválido porque el compilador no puede determinar un tipo a partir de null.

2. Respuesta correcta: B) `hola`

String es inmutable. `concat()` devuelve un nuevo String, pero como no se asigna a ninguna variable, el valor original de 's' no cambia.

3. Respuesta correcta: A) Se genera un String multilínea respetando la sangría mínima común

Los text blocks (JEP 378) permiten literales de String multilínea delimitados por triple comilla doble, y el compilador recorta la sangría incidental común a todas las líneas.

4. Respuesta correcta: B) -128

byte es un tipo de 8 bits con rango -128 a 127. Al incrementar 127 se produce overflow y el valor da la vuelta a -128 (el operador ++ hace un cast implícito).

5. Respuesta correcta: B) `var`

'var', 'yield' y 'record' son 'nombres reservados contextuales', no palabras clave completas; se pueden usar como identificadores en ciertos contextos. Sin embargo, en este listado, 'var' es la única que NO es palabra reservada tradicional (assert sí lo es desde Java 1.4, y sealed/record son contextuales pero suelen listarse aparte). La respuesta correcta según el examen es 'var'.

6. Respuesta correcta: B) 2.0

5 y 2 son ambos int, por lo que la división se realiza como división entera (resultado 2) ANTES de asignarse/convertirse a double. El resultado final es 2.0.

7. Respuesta correcta: A) Permitir mutabilidad y evitar la creación de múltiples objetos al concatenar

StringBuilder es mutable, lo que evita crear múltiples objetos intermedios al hacer concatenaciones repetidas, mejorando el rendimiento.

8. Respuesta correcta: A) Se lanza `UnsupportedOperationException` en tiempo de ejecución

`List.of()` crea una lista inmutable. Cualquier intento de modificarla (`add`, `remove`, `set`) lanza `UnsupportedOperationException` en tiempo de ejecución.

9. Respuesta correcta: B) `false`

Las variables de instancia (campos) numéricas se inicializan a 0, boolean a false, y las de tipo referencia a null, si no se inicializan explícitamente.

10. Respuesta correcta: A) 10100

Los guiones bajos en literales numéricos (desde Java 7) solo son separadores visuales y se ignoran; `10_000` es 10000 y `1_0_0` es 100. El resultado es 10100.

11. Respuesta correcta: B) `true false`

Java cachea instancias de Integer entre -128 y 127. Para 127, a y b apuntan al mismo objeto cacheado (true). Para 200, fuera del rango del caché, se crean objetos distintos (false).

12. Respuesta correcta: A) B

En una expresión aritmética, char se promueve a int. 'A' equivale a 65, +1 es 66, y el cast (char) lo convierte de nuevo al carácter correspondiente: 'B'.

13. Respuesta correcta: B) Se lanza NullPointerException al intentar desempaquetar (unboxing) el valor null

Al asignar un Integer null a un int primitivo, Java intenta desempaquetarlo automáticamente (unboxing), lo cual lanza NullPointerException porque no hay un valor int equivalente a null.

14. Respuesta correcta: B) 5.0

En el operador ternario, si las dos ramas tienen tipos numéricos distintos (int y double), el resultado se promueve al tipo más amplio (double), sin importar cuál rama se elija en tiempo de ejecución. Por eso se imprime 5.0.

15. Respuesta correcta: C) false true

new String("hola") crea un objeto nuevo fuera del pool de Strings, por lo que == (comparación de referencias) da false. equals() compara el contenido, que sí es igual, dando true.

16. Respuesta correcta: A) 00042

El formato %05d indica un entero con ancho mínimo de 5 caracteres, rellenado con ceros a la izquierda: '00042'.

17. Respuesta correcta: A) int x = 5; if (x instanceof Integer) { }

'instanceof' solo puede aplicarse sobre tipos de referencia. 'x' es de tipo primitivo int, por lo que esa línea produce un error de compilación.

18. Respuesta correcta: C) 12

i++ (postfijo) devuelve 5 y luego incrementa i a 6. ++i (prefijo) incrementa i a 7 y devuelve 7. La suma es 5 + 7 = 12.

19. Respuesta correcta: A) 1 7

El AND bit a bit (0101 & 0011 = 0001 = 1) y el OR bit a bit (0101 | 0011 = 0111 = 7) operan sobre la representación binaria de los enteros.

20. Respuesta correcta: A) 3

'var' puede inferir perfectamente tipos de array (aquí int[]) a partir del inicializador, sin ninguna restricción especial.

21. Respuesta correcta: B) Se lanza ArrayStoreException en tiempo de ejecución

Los arrays en Java son covariantes: se permite asignar un String[] a una referencia Object[]. Pero el array conserva su tipo REAL (String[]) en tiempo de ejecución, así que intentar guardar un Integer lanza ArrayStoreException.

22. Respuesta correcta: A) true false

Character.isDigit('7') es true porque '7' es un dígito. Character.isLetter('7') es false porque un dígito no es una letra.

23. Respuesta correcta: B) 7

Math.abs(-7) devuelve 7, y Math.max(3, 7) devuelve el mayor de los dos: 7.

24. Respuesta correcta: A) [Java]

trim() elimina los espacios en blanco al inicio y al final de la cadena, dejando '[Java]'.

25. Respuesta correcta: A) [1, 2, 3]

En un for-each, la variable 'n' es una COPIA del valor de cada elemento; modificarla no afecta al array original. El array permanece [1, 2, 3].

26. Respuesta correcta: A) Como tipo de un parámetro de un método regular (no lambda)

'var' solo puede usarse para inferir el tipo de variables LOCALES con inicializador; no puede usarse en parámetros de métodos regulares, campos de clase ni tipos de retorno.

2. Control de flujo (if, switch, bucles, pattern matching)

27. Respuesta correcta: B) dos

La expresión switch con flechas (->) de Java 14+ evalúa el caso coincidente y no requiere 'break'; con x=2 se selecciona 'dos'.

28. Respuesta correcta: A) TEXTO

Pattern matching for instanceof (Java 16+) permite declarar 's' dentro de la condición. 'texto' tiene longitud 5 (>3), por lo que se imprime en mayúsculas.

29. Respuesta correcta: B) 002021220

Cuando j==1 se salta la impresión con continue, así que para cada i se imprimen j=0 y j=2: 00,02,10,12,20,22 -> '002021220' (concatenado sin separadores: 00 02 10 12 20 22 => '002021220').

30. Respuesta correcta: B) diez\neinte

El switch tradicional tiene 'fall-through': al no haber 'break' después de case 10, la ejecución continúa hacia case 20 e imprime ambos mensajes.

31. Respuesta correcta: A) case Integer i -> System.out.println(i);

El pattern matching en switch permite 'case TipoReferencia variable ->' para tipos de referencia; no aplica directamente a tipos primitivos como int.

32. Respuesta correcta: A) 00

Cuando i=0, j=0 se imprime '00 '. En la siguiente iteración j=1, se cumple la condición y 'break outer' termina AMBOS bucles inmediatamente.

33. Respuesta correcta: B) 5

do-while ejecuta el cuerpo AL MENOS una vez antes de evaluar la condición. Se imprime '5', luego n=4, y la condición (4 > 5) es falsa, así que termina.

34. Respuesta correcta: A) Dentro de un bloque { } de un case, para devolver el valor de la expresión switch

'yield' se usa dentro de bloques de código en un switch expression cuando el case necesita lógica adicional antes de producir el valor resultante.

35. Respuesta correcta: B) 02

i toma los valores 0, 2, 4 (incrementando de 2 en 2). Cuando i llega a 4, el break detiene el bucle ANTES de imprimir ese valor. Se imprimen 0 y 2.

36. Respuesta correcta: B) El compilador exige exhaustividad y puede dar error si faltan casos y no hay default

Las switch expressions deben ser exhaustivas: si no cubren todos los valores del enum, el compilador exige un 'default' (o falla en tiempo de compilación).

37. Respuesta correcta: B) M

El switch clásico admite String como selector desde Java 7, comparando internamente con equals(). Como día es 'MAR', se ejecuta ese case, imprimiendo 'M'.

38. Respuesta correcta: B) medio

$x=5$ no es mayor que 10, así que se evalúa la segunda condición: $5 > 3$ es true, por lo tanto el resultado es 'medio'.

39. Respuesta correcta: B) No, 'break' no puede devolver un valor en un switch expression; se debe usar 'yield'

Cuando un switch expression usa bloques 'case:' con dos puntos, la forma correcta de producir el valor resultante es con 'yield valor;'. 'break valor' no es una sintaxis válida.

40. Respuesta correcta: A) 3

El bucle incrementa i hasta que llega a 3, momento en el cual 'break' detiene el bucle. Se imprime 3.

41. Respuesta correcta: A) ab

LinkedHashMap conserva el orden de inserción. El for-each sobre entrySet() recorre las entradas en ese orden, imprimiendo las claves 'a' y 'b'.

42. Respuesta correcta: B) No, falta cubrir AZUL o agregar un caso default

Un switch expression debe ser exhaustivo. Como el enum Color tiene tres valores y solo se cubren dos (ROJO, VERDE), el compilador exige cubrir AZUL o agregar un 'default'.

43. Respuesta correcta: A) 012

do-while sin llaves aplica solo a la siguiente sentencia. Se imprime i (0), luego se evalúa $++i < 3$ en cada iteración: imprime 0, 1, 2, y se detiene cuando i llega a 3.

44. Respuesta correcta: B) vacío o null

El operador && es de cortocircuito (short-circuit): como 'arr != null' es false, Java no evalúa 'arr.length', evitando un NullPointerException, y el resultado final es false, imprimiendo 'vacío o null'.

45. Respuesta correcta: A) 001020

Cuando $j==1$, 'continue outer' salta directamente a la siguiente iteración del bucle EXTERNO. Solo se imprime cuando $j==0$ para cada i : '00', '10', '20', concatenados: '001020'.

46. Respuesta correcta: A) uno o dos

Al no haber código entre 'case 1:' y 'case 2:', ambos casos comparten el mismo bloque gracias al fall-through, por lo que $x=2$ también ejecuta 'uno o dos'.

47. Respuesta correcta: A) 024

El for solo requiere la condición; la actualización se hace manualmente dentro del cuerpo. i toma los valores 0, 2, 4 antes de que la condición ($i<5$) se vuelva falsa, imprimiendo '024'.

48. Respuesta correcta: B) medio

El switch expression admite múltiples etiquetas separadas por coma en un mismo case. Como 3 coincide con 'case 3, 4', el resultado es 'medio'.

49. Respuesta correcta: B) x

'for(;;)' es una forma válida de crear un bucle sin condición de parada explícita, pero el 'break' dentro del cuerpo lo detiene inmediatamente tras imprimir 'x' una sola vez.

50. Respuesta correcta: B) pequeño

El 'else' siempre se asocia al 'if' más cercano sin llaves, es decir, a 'if ($x > 10$)', no al 'if ($x > 0$)' externo. Como $x=5$ no es mayor que 10, se ejecuta el else, imprimiendo 'pequeño'.

51. Respuesta correcta: A) 15

Los operadores de asignación compuesta (+=) realizan un cast implícito al tipo original, a diferencia de escribir 'b = b + 5' directamente (que no compilaría sin cast explícito entre byte e int).

52. Respuesta correcta: B) Se lanza NullPointerException al desempaquetar flag

La condición del ternario requiere un boolean primitivo. Al desempaquetar (unboxing) el Boolean null, se lanza NullPointerException antes de poder evaluar la expresión.

53. Respuesta correcta: A) b

El 'default' puede colocarse en cualquier posición dentro del switch; solo se usa si ningún case coincide. Como s='b' coincide directamente con 'case "b":', se ejecuta ese bloque, imprimiendo 'b'.

3. Programación orientada a objetos (clases, herencia, records, sealed)

54. Respuesta correcta: A) Un constructor compacto (compact constructor)

Los records permiten un 'constructor compacto' sin lista de parámetros explícita (se usan los mismos nombres definidos en el encabezado del record) para validar o normalizar valores antes de la asignación automática de los campos.

55. Respuesta correcta: A) Que solo las clases listadas pueden implementar la interfaz

Las interfaces/clases 'sealed' (Java 17) restringen qué clases pueden extenderlas o implementarlas, mediante la cláusula 'permits', permitiendo jerarquías cerradas y controladas.

56. Respuesta correcta: B) Guau

Esto demuestra polimorfismo en tiempo de ejecución: aunque la referencia es de tipo Animal, el método invocado es el sobrescrito en Perro (dynamic dispatch).

57. Respuesta correcta: D) Posibilidad de heredar de otra clase concreta

Los records son implícitamente 'final' y siempre extienden java.lang.Record, por lo que NO pueden heredar de otra clase (aunque sí pueden implementar interfaces).

58. Respuesta correcta: B) No compila: Auto debe implementar mover() o ser declarada abstracta también

Una clase concreta que extiende una clase abstracta debe implementar todos los métodos abstractos heredados, o de lo contrario debe declararse también como abstracta.

59. Respuesta correcta: B) protected

'protected' permite acceso desde el mismo paquete y también desde subclases ubicadas en otros paquetes (a través de herencia). El acceso 'default' (sin modificador) solo permite acceso dentro del mismo paquete.

60. Respuesta correcta: B) Sobrescribir el método decir() para resolver el conflicto explícitamente

Cuando dos interfaces aportan métodos default con la misma firma, la clase implementadora DEBE sobrescribir el método para resolver la ambigüedad (puede invocar una versión específica con Interfaz.super.metodo()).

61. Respuesta correcta: A) Ejecutar código antes de cada constructor, útil para lógica común de inicialización

Los bloques de inicialización de instancia (delimitados por { }) se ejecutan cada vez que se crea un objeto, justo antes del cuerpo del constructor, después de la inicialización de campos.

62. Respuesta correcta: A) Sí, es exhaustivo porque cubre los tres valores del enum

El switch expression cubre los tres valores posibles del enum (LUNES, MARTES, MIERCOLES agrupados), por lo que es exhaustivo y no necesita 'default'.

63. Respuesta correcta: B) non-sealed

'non-sealed' es el modificador que se aplica a una subclase directa de una clase/interfaz sellada, para 'reabrir' la jerarquía y permitir que cualquier clase la extienda.

64. Respuesta correcta: B) false

Sin sobrescribir equals(), se hereda la implementación de Object, que compara referencias de memoria. p1 y p2 son objetos distintos, así que el resultado es false.

65. Respuesta correcta: A) A

Los métodos estáticos no participan en polimorfismo de tiempo de ejecución. Se resuelven según el tipo de la referencia en tiempo de COMPILACIÓN (A), no el tipo real del objeto.

66. Respuesta correcta: B) Figura creada seguido de Circulo creado

Aunque una clase abstracta no puede instanciarse directamente, sí puede tener constructores que se invocan desde las subclases mediante super(), en orden: primero el de la superclase, luego el de la subclase.

67. Respuesta correcta: A) 10

Desde Java 8, las interfaces pueden tener métodos estáticos, invocables directamente mediante NombreInterfaz.metodo(), sin necesidad de una implementación.

68. Respuesta correcta: A) Sí, los métodos privados de interfaz permiten reutilizar lógica entre métodos default/estáticos

Desde Java 9, las interfaces pueden declarar métodos privados, usados internamente para compartir lógica entre sus métodos default y estáticos sin exponerla públicamente.

69. Respuesta correcta: A) Sí, compila correctamente: el accessor automático nombre() satisface el método de la interfaz

Los records Sí pueden implementar interfaces. El accessor generado automáticamente (nombre()) cumple con el método abstracto declarado en Nombrable.

70. Respuesta correcta: A) Sí, es la sintaxis correcta para una inner class no estática

Una clase interna NO estática requiere una instancia de la clase externa para crearse, usando la sintaxis 'instanciaExterna.new Interna()'.

71. Respuesta correcta: A) No necesita una instancia de Externa para ser creada

Una clase anidada estática (static nested class) no está ligada a una instancia particular de la clase externa, por lo que puede crearse directamente con 'new Externa.Nido()'.

72. Respuesta correcta: A) Miau

Un enum puede implementar interfaces y cada constante puede tener su propio cuerpo con implementaciones específicas de los métodos. GATO.emitir() devuelve 'Miau'.

73. Respuesta correcta: A) int

metodo(long) en B es una SOBRECARGA, no una sobrescritura (los tipos de parámetro son distintos). Al llamar con el literal int 5, Java prefiere la coincidencia exacta heredada metodo(int) antes de ensanchar a long.

74. Respuesta correcta: B) No compila: no se puede sobrescribir un método final

Un método marcado como 'final' no puede ser sobrescrito por ninguna subclase; intentarlo produce un error de compilación.

75. Respuesta correcta: A) Encadenamiento de constructores (constructor chaining) para reutilizar lógica de inicialización

'this(...)' invoca a otro constructor de la misma clase, permitiendo reutilizar lógica de inicialización común sin duplicar código (constructor chaining).

76. Respuesta correcta: A) 1

Los bloques de inicialización estática se ejecutan una única vez, cuando la clase se carga por primera vez en la JVM, sin importar cuántas instancias se creen después.

77. Respuesta correcta: A) Porque Java permite tipos de retorno covariantes: Perro es un subtipo de Animal

Java permite 'tipos de retorno covariantes' en la sobrescritura: el método sobrescrito puede devolver un subtipo del tipo de retorno original, siempre que sea compatible.

78. Respuesta correcta: A) Producto[Mesa]

System.out.println() invoca implícitamente toString() sobre el objeto. Al estar sobrescrito, se usa la implementación personalizada que devuelve 'Producto[Mesa]'.

79. Respuesta correcta: A) Porque el acceso 'private' se aplica a nivel de CLASE, no de instancia: cualquier método de Cuenta puede acceder a los campos privados de otro objeto Cuenta

En Java, el modificador 'private' restringe el acceso a nivel de clase, no de objeto individual. Cualquier método definido dentro de Cuenta puede acceder a los campos privados de cualquier instancia de Cuenta.

4. Manejo de excepciones

80. Respuesta correcta: B) BCD

La excepción es capturada por el catch (imprime B), luego SIEMPRE se ejecuta finally (imprime C), y como la excepción fue manejada, el flujo continúa e imprime D.

81. Respuesta correcta: A) MiExcepcion extiende Exception (checked), por lo que 'metodo' debe declarar 'throws MiExcepcion' o capturarla

Al extender Exception (no RuntimeException), MiExcepcion es una excepción 'checked'. Cualquier método que la lance debe declararla en su cláusula throws o manejarla con try-catch.

82. Respuesta correcta: B) AutoCloseable

El try-with-resources requiere que el recurso implemente la interfaz AutoCloseable (o Closeable), para que su método close() se invoque automáticamente al final del bloque.

83. Respuesta correcta: B) AIOOBE

Acceder a un índice fuera de rango de un array lanza ArrayIndexOutOfBoundsException, que es capturada por el segundo bloque catch correspondiente.

84. Respuesta correcta: C) NumberFormatException

NumberFormatException extiende IllegalArgumentException que extiende RuntimeException, por lo que es 'unchecked'. Las otras tres son excepciones 'checked' (heredan directamente de Exception).

85. Respuesta correcta: A) Sí, multi-catch con RuntimeException y Error es válido

Un bloque multi-catch (con |) puede combinar cualquier tipo de excepción (checked, unchecked o Error) siempre que no exista relación de herencia directa entre ellas. RuntimeException y Error no están relacionadas jerárquicamente entre sí, así que es válido.

86. Respuesta correcta: B) 2

Cuando finally contiene un 'return', este SIEMPRE sobrescribe cualquier valor de retorno pendiente del bloque try, por lo que el método devuelve 2 (aunque esta práctica se considera un mal hábito de programación).

87. Respuesta correcta: B) En orden inverso al de su declaración

Los recursos se cierran en orden inverso a como fueron abiertos/declarados, similar a una pila (el último recurso abierto es el primero en cerrarse).

88. Respuesta correcta: A) No compila: el catch de ArithmeticException es inalcanzable porque Exception ya lo cubre y está antes

Cuando se ordenan bloques catch, las subclases deben ir ANTES que sus superclases. Como Exception es superclase de ArithmeticException y va primero, el segundo catch es código inalcanzable, lo que produce un error de compilación.

89. Respuesta correcta: B) Throwable

Throwable es la superclase raíz de toda la jerarquía de excepciones y errores en Java; de ella derivan tanto Exception como Error.

90. Respuesta correcta: A) original

getCause() devuelve la excepción original encadenada (la NullPointerException pasada al constructor), cuyo mensaje es 'original'.

91. Respuesta correcta: A) Por el análisis de 'rethrow preciso' (precise rethrow) desde Java 7, que infiere los tipos reales lanzados dentro del try

Desde Java 7, si la variable capturada en el catch es efectivamente final y solo se relanza (throw e), el compilador analiza los tipos concretos que pueden lanzarse dentro del try, permitiendo una cláusula throws más precisa.

92. Respuesta correcta: A) fin

Las aserciones están deshabilitadas por defecto en la JVM; deben activarse explícitamente con la opción -ea. Sin ella, la sentencia 'assert' se ignora por completo.

93. Respuesta correcta: A) Sí, es completamente válido

Un bloque try puede ir acompañado solo de finally (sin catch), siempre que exista al menos uno de los dos (catch o finally).

94. Respuesta correcta: A) capturado: fallo al crear

Una excepción lanzada dentro de un constructor puede capturarse igual que cualquier otra, ya que la creación del objeto ocurre dentro del bloque try.

95. Respuesta correcta: A) StackOverflowError

La recursión infinita agota el espacio de la pila de llamadas (call stack), lanzando un StackOverflowError, que es una subclase de Error, no de Exception.

96. Respuesta correcta: A) no numérico

Integer.parseInt() lanza NumberFormatException cuando la cadena no representa un número entero válido, como es el caso de 'abc'.

97. Respuesta correcta: B) Se lanza ClassCastException en tiempo de ejecución

El objeto real referenciado por 'o' es un String, no un Integer. El cast explícito falla en tiempo de ejecución con ClassCastException.

98. Respuesta correcta: A) Porque extiende RuntimeException, que es una excepción 'unchecked'

Las excepciones que extienden RuntimeException (directa o indirectamente) son 'unchecked', por lo que el compilador no exige declararlas en 'throws' ni capturarlas obligatoriamente.

99. Respuesta correcta: A) La del cuerpo del try; la de close() queda registrada como excepción suprimida (suppressed)

En try-with-resources, si el cuerpo lanza una excepción y luego close() lanza otra, la excepción del cuerpo es la que se propaga como principal, y la de close() se añade como 'suprimida' (accesible con getSuppressed()).

100. Respuesta correcta: B) IllegalStateException("del finally"); la original se pierde

Si el bloque finally lanza una excepción, esta REEMPLAZA a cualquier excepción pendiente del try, incluso perdiendo la información de la original (una práctica generalmente desaconsejada).

101. Respuesta correcta: B) No, porque Runnable.run() no declara 'throws IOException' y esta es checked

El cuerpo de una lambda debe respetar la cláusula throws del método abstracto de la interfaz funcional que implementa. Runnable.run() no declara excepciones checked, por lo que lanzar IOException dentro no compila.

102. Respuesta correcta: A) Sí, siempre que 'recurso' sea final o efectivamente final

Desde Java 9, try-with-resources admite usar variables ya existentes (no solo declaraciones nuevas), siempre que sean final o efectivamente final.

103. Respuesta correcta: B) Se lanza UnsupportedOperationException porque Arrays.asList() devuelve una lista de tamaño fijo

Arrays.asList() devuelve una lista de tamaño fijo respaldada por el array original; admite modificar elementos existentes con set(), pero no agregar ni eliminar.

104. Respuesta correcta: B) Se lanza NullPointerException con el mensaje 'dato no puede ser null'

Objects.requireNonNull() lanza explícitamente una NullPointerException con el mensaje indicado si el argumento es null, útil para validar parámetros de forma temprana y clara.

105. Respuesta correcta: A) Sí, Throwable es superclase tanto de Exception como de Error

Throwable es la superclase de toda la jerarquía, incluyendo Error y Exception. Aunque no es recomendable en la práctica, técnicamente sí se puede capturar un Error con catch(Throwable t).

106. Respuesta correcta: A) Porque en un multi-catch, la variable de excepción es implícitamente final y no puede reasignarse

En un bloque multi-catch (Tipo1 | Tipo2 e), la variable 'e' es implícitamente final; intentar reasignarle un nuevo valor produce un error de compilación.

5. Arrays y Collections Framework

107. Respuesta correcta: B) [1, 3, 5, 8]

Arrays.sort() ordena el array de forma ascendente 'in place' (modifica el array original). Arrays.toString() genera una representación legible.

108. Respuesta correcta: B) Se lanza ConcurrentModificationException

Modificar una colección directamente (con lista.remove) mientras se itera con un Iterator externo (sin usar it.remove()) provoca ConcurrentModificationException en la siguiente llamada a next() o hasNext().

109. Respuesta correcta: C) 11

merge() aplica la función indicada (aquí, suma) al valor existente y al nuevo valor cuando la clave ya existe: $1 + 10 = 11$.

110. Respuesta correcta: B) ArrayList

ArrayList se basa en un array interno redimensionable, lo que permite acceso por índice en tiempo constante $O(1)$, además de mantener el orden de inserción.

111. Respuesta correcta: B) [apple, banana, cherry]

TreeSet mantiene sus elementos ordenados según su orden natural (alfabético para String), independientemente del orden de inserción.

112. Respuesta correcta: A) [3, 1, 2]

ArrayDeque permite insertar en ambos extremos. addFirst(1) -> [1]; addLast(2) -> [1,2]; addFirst(3) -> [3,1,2].

113. Respuesta correcta: B) [3, 2, 1]

Comparator.reverseOrder() ordena los elementos en orden descendente según su orden natural: [3, 2, 1].

114. Respuesta correcta: A) Collections.unmodifiableList(lista)

Collections.unmodifiableList() envuelve la lista original en una vista que lanza UnsupportedOperationException ante intentos de modificación, aunque los cambios en la lista original subyacente siguen reflejándose.

115. Respuesta correcta: A) 2 3

matriz.length devuelve el número de filas (2) y matriz[0].length el número de columnas de la primera fila (3).

116. Respuesta correcta: B) LinkedHashMap mantiene el orden de inserción de las claves, HashMap no garantiza ningún orden

LinkedHashMap añade una lista doblemente enlazada interna que preserva el orden de inserción (o de acceso, si se configura), mientras que HashMap no garantiza ningún orden específico de iteración.

117. Respuesta correcta: A) [1, 3, 5]

removeIf() elimina de la colección todos los elementos que cumplen el predicado dado; aquí elimina los números pares, dejando [1, 3, 5].

118. Respuesta correcta: B) 1

PriorityQueue es por defecto un min-heap: poll() siempre extrae y devuelve el elemento MENOR según el orden natural, en este caso 1.

119. Respuesta correcta: A) 9 1

Collections.max() y Collections.min() recorren la colección y devuelven el mayor y menor elemento según su orden natural, respectivamente: 9 y 1.

120. Respuesta correcta: B) Se lanza UnsupportedOperationException, porque List.copyOf() es immutable

List.copyOf() crea una copia IMMUTABLE e independiente de la colección original; cualquier intento de modificarla lanza UnsupportedOperationException.

121. Respuesta correcta: B) [Al, Bo, Ana]

Se ordena primero por longitud ('Bo' y 'Al' tienen 2 letras, 'Ana' tiene 3). Para desempatar entre 'Bo' y 'Al' (misma longitud), thenComparing usa el orden natural alfabético: 'Al' antes que 'Bo'. Resultado: [Al, Bo, Ana].

122. Respuesta correcta: A) 3 5

`floor(4)` devuelve el mayor elemento menor o igual a 4 (que es 3). `ceiling(4)` devuelve el menor elemento mayor o igual a 4 (que es 5).

123. Respuesta correcta: A) 6

`values()` devuelve una colección con los valores del mapa (1, 2, 3). Al convertirlos a `IntStream` y sumarlos, el resultado es 6.

124. Respuesta correcta: A) [kiwi, fresa, banana]

Ordenando por longitud ascendente: 'kiwi' (4), 'fresa' (5), 'banana' (6). Resultado: [kiwi, fresa, banana].

125. Respuesta correcta: B) 2

Sin sobrescribir `equals()`/`hashCode()`, cada objeto `Punto` se considera distinto (comparación por referencia), por lo que ambas instancias se agregan al `Set`, resultando en tamaño 2.

126. Respuesta correcta: B) [1, 99, 3, 4, 5]

`subList()` devuelve una VISTA respaldada por la lista original, no una copia independiente. Modificar la sublista afecta directamente a la lista original.

127. Respuesta correcta: B) 99

`Arrays.asList()` devuelve una lista respaldada directamente por el array original; modificar el array se refleja inmediatamente en la lista, y viceversa.

128. Respuesta correcta: A) 3 2

`pop()` elimina y devuelve el elemento en el tope de la pila (3). Tras eliminarlo, `peek()` consulta el nuevo tope (2) sin eliminarlo.

129. Respuesta correcta: A) Vector está sincronizado internamente (thread-safe), con menor rendimiento en entornos de un solo hilo

`Vector` es una clase legada que sincroniza internamente sus métodos, garantizando seguridad en entornos multihilo pero con un costo de rendimiento innecesario cuando no se necesita esa sincronización.

130. Respuesta correcta: A) 5

`EnumSet.range()` crea un conjunto con todos los valores del enum entre los dos indicados (inclusive). De `LUN` a `VIE` hay 5 valores.

131. Respuesta correcta: B) Se lanza UnsupportedOperationException

`Collections.emptyList()` devuelve una lista inmutable de tamaño fijo (cero); cualquier intento de modificarla lanza `UnsupportedOperationException`.

132. Respuesta correcta: A) 1

`indexOf()` devuelve el índice de la PRIMERA ocurrencia del elemento buscado. 'b' aparece por primera vez en el índice 1.

133. Respuesta correcta: A) Porque la lista no está ordenada previamente, y binarySearch requiere una lista ordenada para funcionar correctamente

`Collections.binarySearch()` asume que la lista ya está ordenada según el orden natural (o el `Comparator` dado). Si no lo está, el resultado no está garantizado y puede ser incorrecto.

6. Java Stream API y expresiones lambda

134. Respuesta correcta: A) 2

El filtro selecciona los números pares (2 y 4), y count() devuelve la cantidad de elementos restantes en el stream: 2.

135. Respuesta correcta: B) Lanza IllegalStateException porque un stream ya consumido no puede reutilizarse

Un Stream solo puede consumirse (recorrerse por completo) UNA vez; intentar operar de nuevo sobre él tras una operación terminal lanza IllegalStateException.

136. Respuesta correcta: A) ABC

reduce() acumula un resultado combinando el acumulador con la primera letra de cada nombre: "" + 'A' -> 'A' + 'B' -> 'AB' + 'C' -> 'ABC'.

137. Respuesta correcta: A) R apply(T t)

Function<T, R> define el método abstracto R apply(T t), que transforma un valor de tipo T en uno de tipo R. accept() pertenece a Consumer, test() a Predicate, y get() a Supplier.

138. Respuesta correcta: A) 5

max() con el comparador natural de enteros devuelve un Optional con el valor máximo del stream, que es 5.

139. Respuesta correcta: A) [1, 2, 4, 8]

Stream.iterate genera un stream infinito aplicando repetidamente la función (duplicar), y limit(4) toma los primeros 4 elementos: 1, 2, 4, 8.

140. Respuesta correcta: A) [2, 4]

partitioningBy() divide el stream en dos listas según el predicado: la clave 'true' contiene los que cumplen la condición (pares): [2, 4].

141. Respuesta correcta: A) map() transforma cada elemento en otro elemento; flatMap() transforma cada elemento en un stream y luego los 'aplana' en uno solo

map() aplica una función de transformación 1 a 1. flatMap() se usa cuando cada elemento produce un stream (por ejemplo, una lista de listas), y combina todos esos streams en uno solo aplanado.

142. Respuesta correcta: A) 12345

rangeClosed(1, 5) genera un IntStream con los valores 1, 2, 3, 4, 5 (incluyendo el límite superior), a diferencia de range() que lo excluye.

143. Respuesta correcta: A) 7

BinaryOperator<T> es una especialización de BiFunction<T,T,T> con el método apply(T,T) que devuelve T; aquí simplemente suma los dos argumentos: 3+4=7.

144. Respuesta correcta: A) [uno, dos, tres]

Collectors.joining(delimitador, prefijo, sufijo) concatena los elementos con el delimitador dado, y añade el prefijo y sufijo indicados al resultado final.

145. Respuesta correcta: C) sorted()

sorted() es una operación intermedia que devuelve otro Stream y se ejecuta de forma perezosa. forEach, collect y count son operaciones terminales que sí disparan la ejecución del pipeline.

146. Respuesta correcta: A) Es una operación intermedia usada típicamente para depuración, que no modifica el stream

peek() es una operación intermedia que permite observar los elementos que pasan por el stream (usualmente para depuración) sin transformarlos ni consumirlos.

147. Respuesta correcta: A) 5

Collectors.toMap() construye un Map usando la primera función lambda para la clave y la segunda para el valor. 'Bruno' tiene 5 caracteres.

148. Respuesta correcta: A) xxx

Stream.generate() crea un stream infinito a partir de un Supplier. limit(3) trunca el stream a los primeros 3 elementos antes de que se ejecute forEach, evitando el bucle infinito.

149. Respuesta correcta: A) true true

allMatch() verifica que TODOS los elementos cumplan la condición (todos son pares: true). anyMatch() verifica si AL MENOS UNO cumple la condición (8 > 6: true).

150. Respuesta correcta: A) 1 3

Stream.of(arr) con un array de tipo primitivo trata el array COMPLETO como un único elemento (produciendo un stream de tamaño 1). Arrays.stream(arr) sí descompone el array en sus 3 elementos individuales.

151. Respuesta correcta: B) 1

Agrupando por longitud: 'uno'(3), 'dos'(3), 'tres'(4), 'cuatro'(6), 'cinco'(5). Solo 'tres' tiene longitud 4, así que porLongitud.get(4) es 1.

152. Respuesta correcta: A) orElseGet() no ejecuta el Supplier si el valor ya está presente, evitando cálculos innecesarios; orElse() siempre evalúa su argumento

orElse(valor) siempre evalúa su argumento (aunque no se use), mientras que orElseGet(supplier) solo invoca el Supplier si el Optional está vacío, siendo más eficiente para cálculos costosos.

153. Respuesta correcta: A) bananakiwiuva

Ordenando por longitud de forma descendente: 'banana'(6), 'kiwi'(4), 'uva'(3). El resultado impreso es 'bananakiwiuva'.

154. Respuesta correcta: A) n1,n2,n3

IntStream.range(1,4) genera los valores 1, 2, 3 (excluyendo el límite superior). mapToObj() los transforma en Strings 'n1','n2','n3', unidos con coma.

155. Respuesta correcta: A) 4.0

Collectors.averagingInt() calcula el promedio de los valores como Double. (2+4+6)/3 = 4.0.

156. Respuesta correcta: A) 1234

Stream.concat() combina dos streams en uno solo, respetando el orden: primero los elementos de s1, luego los de s2.

157. Respuesta correcta: A) 2

distinct() usa equals()/hashCode(). Los records generan estos métodos automáticamente basados en sus componentes, por lo que los dos Punto(1,1) se consideran iguales y se cuentan una sola vez: quedan 2 elementos distintos.

158. Respuesta correcta: A) 8

andThen() aplica primero la función original y luego la siguiente: (3+1)=4, luego 4*2=8.

159. Respuesta correcta: A) true

negate() invierte el resultado del predicado original. Como 3 no es par (esPar.test(3) es false), esImpar.test(3) da true.

160. Respuesta correcta: A) 3

filter() selecciona los elementos mayores a 2, y findFirst() es una operación de cortocircuito que devuelve el primero que cumple la condición sin procesar el resto: 3.

7. Concurrency

161. Respuesta correcta: B) Se lanza IllegalStateException en la segunda llamada a start()

Un hilo (Thread) solo puede iniciarse una vez con start(); llamarlo una segunda vez sobre el mismo objeto Thread lanza IllegalStateException.

162. Respuesta correcta: B) Permite que las tareas ya enviadas terminen, pero rechaza nuevas tareas

shutdown() inicia un apagado ordenado: las tareas ya enviadas se completan, pero no se aceptan tareas nuevas. Para forzar la detención inmediata se usaría shutdownNow().

163. Respuesta correcta: B) Que solo un hilo a la vez pueda ejecutar este método sobre la misma instancia, evitando condiciones de carrera

'synchronized' en un método de instancia usa el monitor del objeto ('this') para asegurar exclusión mutua: solo un hilo puede ejecutar ese método (o cualquier otro sincronizado sobre el mismo objeto) a la vez.

164. Respuesta correcta: A) Callable puede devolver un valor y lanzar excepciones checked; Runnable no devuelve valor

Callable<V> define V call() throws Exception, permitiendo devolver un resultado y lanzar excepciones checked. Runnable define void run() sin retorno y sin declarar excepciones checked.

165. Respuesta correcta: C) 2

AtomicInteger provee operaciones atómicas thread-safe sin necesidad de bloqueos explícitos. incrementAndGet() incrementa y devuelve el nuevo valor; tras dos llamadas, el contador vale 2.

166. Respuesta correcta: A) 10

future.get() bloquea hasta obtener el resultado de la tarea Callable enviada. El cálculo 5+5 da 10. (Nota: get() lanza ExecutionException/InterruptedException checked que deberían manejarse o declararse, pero el resultado lógico es 10).

167. Respuesta correcta: A) Dos o más hilos quedan bloqueados esperando indefinidamente recursos que el otro posee

Un deadlock ocurre cuando dos o más hilos se bloquean mutuamente esperando recursos (locks) que están retenidos por el otro, y ninguno puede avanzar.

168. Respuesta correcta: B) Se ejecuta sin error, imprimiendo 2 (el bucle solo itera sobre el snapshot original)

CopyOnWriteArrayList crea una copia interna del array en cada modificación; el iterador opera sobre una instantánea (snapshot) tomada al crear el iterador, por lo que no lanza excepción aunque se modifique la lista durante la iteración.

169. Respuesta correcta: A) Hacer que uno o más hilos esperen hasta que un conjunto de operaciones realizadas en otros hilos se complete

CountDownLatch permite que uno o más hilos bloqueen su ejecución con await() hasta que un contador (inicializado en un número) llegue a cero mediante llamadas a countDown() desde otros hilos.

170. Respuesta correcta: B) Thread.stop()

Thread.stop() está deprecado porque termina el hilo de forma abrupta, pudiendo dejar recursos o estructuras de datos compartidas en un estado inconsistente. Se recomienda usar mecanismos de interrupción cooperativa (interrupt()).

171. Respuesta correcta: A) Que los cambios hechos por un hilo sean visibles inmediatamente para otros hilos, evitando cachés locales desactualizadas

'volatile' garantiza visibilidad entre hilos (happens-before), pero NO garantiza atomicidad en operaciones compuestas como i++, que requieren sincronización adicional.

172. Respuesta correcta: A) Para garantizar que el lock se libere incluso si ocurre una excepción en la sección crítica

Si no se libera el lock en un finally, una excepción en la sección crítica dejaría el lock retenido indefinidamente, provocando un posible deadlock para otros hilos.

173. Respuesta correcta: A) Estar dentro de un bloque synchronized sobre ese mismo objeto, o se lanza IllegalMonitorStateException

wait() (y notify()/notifyAll()) solo pueden invocarse dentro de un bloque sincronizado sobre el monitor de ese objeto; de lo contrario se lanza IllegalMonitorStateException.

174. Respuesta correcta: A) Nada garantizado de forma determinista: es solo una sugerencia al planificador del sistema operativo

La prioridad de un hilo es solo una sugerencia (hint) para el planificador; el comportamiento real depende del sistema operativo y no está garantizado por la especificación de Java.

175. Respuesta correcta: A) Mantener una copia independiente de la variable para cada hilo que la use

ThreadLocal provee una variable independiente para cada hilo que la accede, evitando condiciones de carrera sin necesidad de sincronización explícita.

176. Respuesta correcta: A) Ejecuta todas las tareas y bloquea hasta que TODAS terminen, devolviendo sus Future

invokeAll() envía todas las tareas Callable al executor y bloquea el hilo llamador hasta que TODAS hayan finalizado, devolviendo una lista de Future con sus resultados.

177. Respuesta correcta: A) Permite que la clase siga pudiendo extender otra clase, ya que Java no permite herencia múltiple

Al implementar Runnable (en vez de extender Thread), la clase conserva la posibilidad de extender otra clase distinta, dado que Java no permite herencia múltiple de clases.

178. Respuesta correcta: A) 10

supplyAsync() ejecuta el Supplier de forma asíncrona (produciendo 5), y thenApply() transforma ese resultado ($5 \times 2 = 10$) una vez disponible.

179. Respuesta correcta: A) Sobre el objeto Class asociado a la clase Contador, no sobre una instancia particular

Un método estático synchronized usa como monitor el objeto Class de la clase (Contador.class), compartido por todas las instancias y llamadas estáticas.

180. Respuesta correcta: A) No compila porque Thread.sleep() lanza InterruptedException (checked) que debe manejarse o declararse

Thread.sleep() declara 'throws InterruptedException', una excepción checked. El método que la invoca debe capturarla o declarar también esa excepción en su firma.

181. Respuesta correcta: A) Ejecuta la tarea repetidamente cada cierto intervalo de tiempo fijo, comenzando de inmediato

scheduleAtFixedRate() programa la ejecución repetida de una tarea con un intervalo de tiempo fijo entre inicios de ejecución, comenzando tras el retraso inicial indicado (0 en este caso).

182. Respuesta correcta: A) Permite mayor concurrencia al usar particionamiento interno con bloqueos más finos, en lugar de bloquear todo el mapa en cada operación

ConcurrentHashMap divide internamente sus datos para permitir que múltiples hilos operen simultáneamente sobre distintas partes del mapa, logrando mucho mayor concurrencia que bloquear el mapa completo en cada acceso.

183. Respuesta correcta: A) La JVM puede terminar el proceso aunque este hilo siga en ejecución, ya que no impide la salida del programa

Los hilos daemon se usan para tareas de soporte en segundo plano; la JVM finaliza el programa en cuanto solo quedan hilos daemon en ejecución, sin esperar a que terminen.

184. Respuesta correcta: A) Que como máximo 2 hilos accedan simultáneamente a la sección protegida

Un Semaphore limita el número de hilos que pueden acceder concurrentemente a un recurso, según la cantidad de 'permisos' disponibles (2 en este caso).

185. Respuesta correcta: A) El valor final puede ser menor al esperado debido a una condición de carrera (race condition), ya que valor++ no es atómico

valor++ implica leer, incrementar y escribir; si dos hilos lo hacen simultáneamente sin sincronización, algunos incrementos pueden perderse, resultando en un valor final menor al esperado.

186. Respuesta correcta: C) PAUSED

Los estados válidos de Thread.State son NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING y TERMINATED. 'PAUSED' no existe como estado de hilo en Java.

187. Respuesta correcta: A) Bloquea el hilo actual hasta que el hilo 't' finalice su ejecución

join() hace que el hilo que lo invoca (en este caso, el hilo principal) espere hasta que el hilo 't' termine su ejecución antes de continuar.

8. Java I/O y NIO.2

188. Respuesta correcta: A) datos/archivo.txt

Path.of() (o Paths.get()) construye una ruta a partir de los segmentos indicados, usando el separador del sistema operativo (en Unix, '/').

189. Respuesta correcta: A) readAllLines() carga todo el archivo en memoria como una List<String>; lines() devuelve un Stream perezoso, más adecuado para archivos grandes

Files.readAllLines() lee todo el contenido de una vez en memoria, mientras que Files.lines() devuelve un Stream<String> de lectura perezosa, ideal para archivos grandes ya que no carga todo en memoria simultáneamente.

190. Respuesta correcta: A) Un patrón decorador: BufferedReader envuelve a FileReader para añadir buffering

La API de I/O clásica de Java usa el patrón decorador: clases como BufferedReader 'envuelven' a otro Reader (FileReader) para añadirle funcionalidad extra, en este caso, lectura eficiente por líneas con buffer.

191. Respuesta correcta: B) Serializable

Una clase debe implementar la interfaz marcador Serializable para que sus instancias puedan escribirse con ObjectOutputStream y leerse luego con ObjectInputStream.

192. Respuesta correcta: A) Permite sobrescribir el archivo destino si ya existe

REPLACE_EXISTING indica a Files.copy() que, si el archivo destino ya existe, debe sobrescribirse en lugar de lanzar FileAlreadyExistsException.

193. Respuesta correcta: A) toAbsolutePath()

toAbsolutePath() convierte una ruta relativa en una ruta absoluta basada en el directorio de trabajo actual. resolve() combina rutas, normalize() elimina elementos redundantes (como '..'), y relativize() calcula una ruta relativa entre dos rutas.

194. Respuesta correcta: A) Recorre recursivamente el árbol de directorios a partir de la ruta dada, devolviendo un Stream<Path>

Files.walk() recorre recursivamente el árbol de directorios comenzando en la ruta indicada, devolviendo un Stream<Path> perezoso con todos los archivos y subdirectorios encontrados.

195. Respuesta correcta: B) transient (modificador, no anotación)

El modificador 'transient' marca un campo para que sea excluido explícitamente del proceso de serialización; al deserializar, ese campo recupera su valor por defecto (0, null, false, etc.).

196. Respuesta correcta: A) false

Files.exists() devuelve false si el archivo o directorio en la ruta dada no existe, sin lanzar ninguna excepción.

197. Respuesta correcta: A) Los crea automáticamente junto con 'c'

Files.createDirectories() crea todos los directorios intermedios necesarios que no existan, a diferencia de Files.createDirectory() que requiere que el padre ya exista.

198. Respuesta correcta: A) Crea el archivo y escribe el contenido indicado

Por defecto, Files.write() usa las opciones CREATE y TRUNCATE_EXISTING: crea el archivo si no existe, o lo sobrescribe si ya existe.

199. Respuesta correcta: A) Habilita el modo append, agregando contenido al final del archivo sin sobrescribirlo

El segundo parámetro booleano del constructor de FileWriter indica el modo 'append': true agrega al final del archivo existente, false (por defecto) lo sobrescribe.

200. Respuesta correcta: A) Ofrece métodos convenientes como println(), print() y printf() para distintos tipos de datos

PrintWriter decora otro Writer añadiendo métodos convenientes (println, print, printf) similares a los de System.out, facilitando la escritura formateada.

201. Respuesta correcta: A) Lanza NoSuchFileException

A diferencia de Files.deleteIfExists() (que devuelve false sin error si el archivo no existe), Files.delete() lanza NoSuchFileException si el archivo no está presente.

202. Respuesta correcta: A) java/App.java

relativize() calcula la ruta relativa necesaria para ir desde 'base' hasta 'destino': 'java/App.java'.

203. Respuesta correcta: A) /datos/proyectos/app.txt

resolve() combina la ruta base con la ruta relativa dada, produciendo '/datos/proyectos/app.txt'.

204. Respuesta correcta: A) Identificar de forma consistente la versión de la clase durante la deserialización, evitando InvalidClassException si la clase cambia

serialVersionUID ayuda a verificar la compatibilidad entre la versión de la clase usada al serializar y la usada al deserializar, evitando InvalidClassException si difieren y no se declaró explícitamente.

205. Respuesta correcta: A) Porque readObject() devuelve Object de forma genérica, sin importar el tipo real serializado

El método readObject() de ObjectInputStream tiene firma 'Object readObject()', por lo que siempre requiere un cast explícito al tipo real esperado.

206. Respuesta correcta: A) Para leer entradas interactivas del usuario desde la terminal, incluyendo contraseñas ocultas con readPassword()

java.io.Console provee acceso a la consola asociada con la JVM, útil para leer entradas de usuario de forma interactiva, incluyendo contraseñas que no se muestran en pantalla.

207. Respuesta correcta: A) Primero b, luego a (orden inverso a la declaración)

En try-with-resources con múltiples recursos, se cierran en orden INVERSO al de su declaración: el último recurso abierto es el primero en cerrarse.

208. Respuesta correcta: A) Usa el separador de línea correcto según el sistema operativo (line.separator)

newLine() usa el separador de línea propio de la plataforma (definido en la propiedad 'line.separator'), lo que garantiza portabilidad entre sistemas operativos.

209. Respuesta correcta: A) El tamaño del archivo en bytes

Files.size() devuelve el tamaño del archivo en bytes, tal como está almacenado en el sistema de archivos.

210. Respuesta correcta: A) Lista los archivos y directorios directamente contenidos en la ruta dada (sin recursividad)

DirectoryStream provee una forma eficiente de iterar sobre las entradas de UN SOLO nivel de un directorio, sin recorrer subdirectorios de forma recursiva.

211. Respuesta correcta: A) Permite leer y escribir en cualquier posición del archivo usando seek(), sin tener que procesar secuencialmente

RandomAccessFile permite mover el puntero de posición del archivo libremente con seek(), habilitando acceso de lectura/escritura en cualquier punto, a diferencia del acceso puramente secuencial de otros flujos.

212. Respuesta correcta: A) Actúa como puente (bridge) entre un flujo de bytes (InputStream) y un flujo de caracteres (Reader), aplicando una codificación de texto

InputStreamReader convierte bytes (de un InputStream como System.in) en caracteres (Reader), aplicando la codificación de caracteres especificada o la por defecto del sistema.

213. Respuesta correcta: A) Mueve (renombra) el archivo origen.txt a destino.txt, sobrescribiendo si ya existe

Files.move() mueve (o renombra) un archivo de una ruta a otra; REPLACE_EXISTING indica que se sobrescriba el destino si ya existe.

214. Respuesta correcta: A) Se lanza NotSerializableException en tiempo de ejecución porque Conexion no implementa Serializable

Cuando un objeto Serializable contiene una referencia a un objeto de una clase que NO implementa Serializable (y no es transient), se lanza NotSerializableException al intentar escribirlo.

9. JDBC (Java Database Connectivity)

215. Respuesta correcta: A) Porque previene inyección SQL y permite reutilizar el plan de ejecución con parámetros

PreparedStatement usa parámetros vinculados (placeholders '?') que son tratados como datos, no como código SQL, previniendo inyección SQL, además de permitir que la base de datos reutilice el plan de ejecución.

216. Respuesta correcta: A) next()

ResultSet.next() mueve el cursor a la siguiente fila y devuelve true si existe una fila válida, o false si se llegó al final del conjunto de resultados.

217. Respuesta correcta: A) Manejo manual de transacciones: desactivar auto-commit, confirmar o revertir cambios explícitamente

Al desactivar autoCommit, el desarrollador controla manualmente cuándo confirmar (commit) o revertir (rollback) un conjunto de operaciones como una única unidad transaccional.

218. Respuesta correcta: A) Gestionar un registro de drivers JDBC y establecer conexiones a bases de datos según la URL proporcionada

DriverManager mantiene un registro de los drivers JDBC cargados y, mediante getConnection(url, ...), selecciona el driver apropiado según el prefijo de la URL para establecer la conexión.

219. Respuesta correcta: A) Ejecutar múltiples sentencias SQL en un solo viaje de red, mejorando el rendimiento

El procesamiento por lotes (batch) agrupa varias sentencias para enviarlas juntas a la base de datos, reduciendo la sobrecarga de comunicación red/servidor y mejorando el rendimiento en inserciones o actualizaciones masivas.

220. Respuesta correcta: A) CallableStatement se usa específicamente para invocar procedimientos almacenados

CallableStatement extiende PreparedStatement y está diseñado específicamente para invocar procedimientos almacenados (stored procedures) en la base de datos, incluyendo soporte para parámetros de salida (OUT).

221. Respuesta correcta: A) jdbc:mysql://host:puerto/basededatos

Las URLs JDBC siguen el formato 'jdbc:subprotocolo://host:puerto/basededatos', donde el subprotocolo identifica el tipo de base de datos (mysql, postgresql, oracle, etc.).

222. Respuesta correcta: A) Garantiza que conn.close() se invoque automáticamente, liberando recursos incluso si ocurre una excepción

Connection implementa AutoCloseable, así que try-with-resources garantiza que la conexión se cierre automáticamente al final del bloque, incluso ante una excepción.

223. Respuesta correcta: A) Se puede acceder por nombre de columna o por índice (basado en 1, no en 0)

JDBC permite acceder a las columnas de un ResultSet tanto por su nombre (getInt("id")) como por su posición numérica, la cual comienza en 1 (no en 0 como los arrays).

224. Respuesta correcta: A) Checked, extiende directamente de Exception

SQLException es una excepción 'checked' (extiende Exception directamente), por lo que debe manejarse con try-catch o declararse en la cláusula throws del método.

225. Respuesta correcta: A) Inyección SQL, ya que 'entradaUsuario' se concatena directamente sin sanitizar ni usar parámetros

Concatenar directamente entradas del usuario en una sentencia SQL permite que un atacante inyecte código SQL malicioso (por ejemplo, ' OR '1'='1). Se debe usar PreparedStatement con parámetros vinculados.

226. Respuesta correcta: A) TYPE_FORWARD_ONLY: solo permite avanzar con next(), no retroceder

Por defecto, un ResultSet es de tipo TYPE_FORWARD_ONLY, permitiendo únicamente avanzar secuencialmente con next(); para navegación bidireccional se debe solicitar explícitamente otro tipo al crear el Statement.

227. Respuesta correcta: A) Se puede agotar el pool de conexiones disponibles, causando fugas de recursos en la aplicación

Las conexiones JDBC son recursos limitados y costosos; no cerrarlas explícitamente puede agotar el pool disponible, causando fallos en la aplicación por falta de conexiones libres.

228. Respuesta correcta: A) El número de filas afectadas por cada sentencia del lote (o un código especial si no se puede determinar)

executeBatch() devuelve un array donde cada posición indica el número de filas afectadas por la sentencia correspondiente del lote, en el mismo orden en que fueron agregadas con addBatch().

229. Respuesta correcta: A) Para obtener valores generados automáticamente por la base de datos, como claves primarias autoincrementales, tras una inserción

Esta combinación permite recuperar valores generados automáticamente por la base de datos (como un ID autoincremental) inmediatamente después de una operación INSERT.

230. Respuesta correcta: A) No, los drivers JDBC 4.0+ se registran automáticamente mediante el mecanismo ServiceLoader

Desde JDBC 4.0, los drivers presentes en el classpath se registran automáticamente al iniciar la aplicación, gracias al mecanismo de descubrimiento de servicios (ServiceLoader), haciendo innecesario el Class.forName() manual.

231. Respuesta correcta: A) Qué tan visibles son los cambios no confirmados de otras transacciones concurrentes para la transacción actual

El nivel de aislamiento (isolation level) determina qué fenómenos de concurrencia (lecturas sucias, no repetibles, fantasma) pueden ocurrir entre transacciones simultáneas.

232. Respuesta correcta: A) Metadatos sobre la estructura del resultado, como número de columnas, sus nombres y tipos

ResultSetMetaData permite consultar la estructura del resultado de una consulta (número de columnas, nombres, tipos de datos) sin conocerla de antemano en tiempo de compilación.

233. Respuesta correcta: A) Navegar en ambas direcciones (adelante y atrás) e ir a posiciones específicas como first()/last()

TYPE_SCROLL_INSENSITIVE permite un cursor desplazable en ambas direcciones, habilitando métodos como first(), last(), absolute() y relative(), a diferencia del cursor por defecto (solo hacia adelante).

234. Respuesta correcta: A) Permite configurar pooling de conexiones y desacoplar la configuración de la conexión del código, apto para servidores de aplicaciones

DataSource desacopla la obtención de conexiones de los detalles concretos del driver, y suele integrarse con mecanismos de pooling gestionados por el contenedor o framework de la aplicación.

235. Respuesta correcta: A) Para verificar si el último valor leído era NULL en la base de datos (ya que getInt() devuelve 0 tanto para NULL como para 0 real)

Los métodos getX() de tipos primitivos devuelven un valor por defecto (como 0) cuando la columna es NULL en la base de datos. wasNull() permite distinguir si ese 0 representa un valor real o un NULL.

236. Respuesta correcta: A) El número de filas afectadas por la operación

executeUpdate() se usa para sentencias INSERT, UPDATE o DELETE, y devuelve un int con el número de filas afectadas por la operación.

237. Respuesta correcta: A) Revertir parcialmente una transacción hasta un punto intermedio, sin deshacer todo lo confirmado previamente

Un Savepoint marca un punto intermedio dentro de una transacción, permitiendo hacer rollback solo hasta ese punto en lugar de deshacer toda la transacción completa.

238. Respuesta correcta: A) Para asignar explícitamente un valor NULL de SQL a un parámetro, indicando el tipo SQL correspondiente

setNull() asigna explícitamente NULL a un parámetro de un PreparedStatement, indicando también el tipo SQL (Types.INTEGER, Types.VARCHAR, etc.) para que el driver lo interprete correctamente.

239. Respuesta correcta: A) Reutilizar un conjunto de conexiones ya establecidas para evitar el costo de abrir y cerrar conexiones repetidamente

Un pool de conexiones mantiene un conjunto de conexiones abiertas y reutilizables, evitando el costoso proceso de crear y destruir conexiones para cada operación de la aplicación.

240. Respuesta correcta: A) absolute(3) posiciona el cursor en la fila 3; relative(-1) lo mueve una posición hacia atrás desde la posición actual

absolute(n) posiciona el cursor en la fila absoluta n; relative(n) mueve el cursor n posiciones relativas a la posición actual (negativo para retroceder). Ambos requieren un ResultSet desplazable (scrollable).

241. Respuesta correcta: A) Para asegurar que ambos recursos se cierren automáticamente en el orden correcto, evitando fugas de recursos de la base de datos

Tanto Statement como ResultSet implementan AutoCloseable; declararlos juntos en try-with-resources asegura que ambos se cierren automáticamente (ResultSet primero, luego Statement) al finalizar el bloque.

10. Código seguro (secure coding)

242. Respuesta correcta: A) Inmutabilidad: campos privados y finales, clase final, sin setters

La clase es 'final' (no se puede extender), sus campos son 'private final' (no pueden reasignarse ni accederse directamente desde fuera), y no expone setters, garantizando que su estado no cambie tras la construcción: es inmutable.

243. Respuesta correcta: A) Los genéricos proporcionan seguridad de tipos en tiempo de compilación, evitando ClassCastException en tiempo de ejecución

Usar genéricos permite que el compilador detecte errores de tipo en tiempo de compilación, evitando que se introduzcan elementos de tipo incorrecto que causarían ClassCastException más adelante en tiempo de ejecución.

244. Respuesta correcta: A) Validación de entradas (input validation) antes de procesarlas

Validar que los datos de entrada sean correctos (no nulos, no vacíos) antes de procesarlos es una práctica fundamental de codificación segura para evitar comportamientos inesperados o vulnerabilidades.

245. Respuesta correcta: A) Declarar los campos como private y exponerlos solo mediante métodos controlados (getters, sin setters si no es necesario)

Encapsular el estado con campos privados y exponer solo lo necesario a través de métodos públicos controlados reduce la posibilidad de que código externo modifique el estado interno de forma no controlada.

246. Respuesta correcta: A) Reduce el riesgo de fugas de recursos (resource leaks) al garantizar el cierre automático, incluso si ocurre una excepción

try-with-resources garantiza que el método close() se invoque automáticamente al final del bloque (incluso si se produce una excepción), reduciendo drásticamente el riesgo de fugas de recursos por errores humanos en el manejo manual.

247. Respuesta correcta: A) Permite que código externo modifique el estado interno sin validación, rompiendo invariantes de la clase

Los campos públicos y mutables permiten que cualquier código externo los modifique directamente, sin pasar por validaciones, lo cual puede romper las invariantes internas de la clase y generar estados inconsistentes o vulnerables.

248. Respuesta correcta: A) Para evitar que código externo modifique el estado interno del objeto a través de una referencia mutable compartida (Date es mutable)

Date es mutable. Sin copias defensivas, tanto quien construye el objeto como quien recibe la fecha por el getter podrían modificar la fecha interna a través de su propia referencia, rompiendo la inmutabilidad de Evento.

249. Respuesta correcta: A) Crear una copia inmutable interna, protegiendo la clase de modificaciones externas posteriores a la lista original pasada al constructor

List.copyOf() crea una copia inmutable e independiente; aunque el llamador modifique la lista original después de construir el Grupo, el estado interno de Grupo permanece intacto.

250. Respuesta correcta: A) Expone una referencia directa al array interno, permitiendo que código externo lo modifique sin control

Los arrays son objetos mutables. Devolver la referencia directa permite que el código externo modifique el array interno (por ejemplo, inventario.getCantidades()[0] = -1), rompiendo la encapsulación.

251. Respuesta correcta: A) Principio de mínimo privilegio (least privilege), reduciendo la superficie de ataque de la clase

El principio de mínimo privilegio recomienda otorgar el nivel de acceso mínimo necesario, reduciendo así las posibilidades de uso indebido o manipulación no autorizada del código.

252. Respuesta correcta: A) Termina abruptamente toda la JVM, afectando a la aplicación completa que use la librería, en lugar de manejar el error localmente

System.exit() termina el proceso completo de la JVM. Una librería que lo invoque puede tumbar inesperadamente toda la aplicación que la use, en lugar de dejar que el código llamador decida cómo manejar el problema.

253. Respuesta correcta: A) Porque un flujo de datos serializado manipulado externamente podría reconstruir un objeto con un estado inválido si no se valida

Un atacante podría modificar manualmente los bytes de un objeto serializado. Validar el estado dentro de readObject() protege contra la reconstrucción de objetos con invariantes rotas.

254. Respuesta correcta: A) Porque un char[] puede sobrescribirse y limpiarse explícitamente de memoria, mientras que un String inmutable puede permanecer en memoria (posiblemente en el pool) hasta que el recolector de basura actúe

Al ser inmutable, un String con una contraseña puede quedar en memoria más tiempo del deseado (por ejemplo, si termina en el pool de Strings). Un char[] permite sobrescribir explícitamente sus posiciones tras usarlo.

255. Respuesta correcta: A) El objeto puede volverse 'inencontrable' en el mapa porque su hashCode cambió y ya no coincide con el bucket original donde fue almacenado

HashMap ubica las entradas según el hashCode calculado al momento de insertarlas. Si el objeto clave muta y su hashCode cambia, buscarlo después (con el nuevo hashCode) no encontrará el bucket original, efectivamente 'perdiendo' la entrada.

256. Respuesta correcta: A) Evita que código malicioso extienda la clase y sobrescriba métodos para alterar su comportamiento esperado

Una clase 'final' no puede ser extendida, evitando que una subclase maliciosa sobrescriba métodos críticos (como validaciones de seguridad) para alterar su comportamiento esperado.

257. Respuesta correcta: A) Validación de formato de entrada mediante una expresión regular antes de aceptar el dato

Validar que una entrada cumpla un formato esperado (mediante regex u otro mecanismo) antes de aceptarla es una práctica fundamental de validación de entradas.

258. Respuesta correcta: A) setAccessible(true) puede saltarse la encapsulación normal (private), permitiendo modificar campos que deberían estar protegidos

La reflexión con setAccessible(true) puede saltarse los modificadores de acceso normales de Java, permitiendo leer o modificar campos privados que deberían estar protegidos por encapsulación.

259. Respuesta correcta: A) Expone información sensible (como la clave) en mensajes de log o consola, lo que puede filtrarse a través de registros

Registrar credenciales (como contraseñas) en mensajes de log o consola es una práctica insegura, ya que esa información puede quedar expuesta en archivos de registro accesibles.

260. Respuesta correcta: A) Permite construir objetos complejos paso a paso, garantizando que el objeto final sea inmutable y consistente una vez creado

El patrón Builder permite ensamblar un objeto complejo de forma flexible, mientras que el constructor privado de Config asegura que, una vez construido, el objeto resultante sea inmutable y no pueda alterarse.

261. Respuesta correcta: A) Cualquier clase puede modificar la lista compartida globalmente sin control ni validación, incluyendo desde hilos concurrentes sin sincronización

Un campo estático público y mutable es accesible y modificable desde cualquier parte del código sin control, y además puede sufrir condiciones de carrera si se accede concurrentemente sin sincronización.

262. Respuesta correcta: A) Es una copia superficial (shallow copy): el clon comparte la MISMA referencia a la lista 'items', por lo que modificar una afecta a la otra

super.clone() realiza una copia superficial: los campos de tipo referencia (como 'items') se copian como referencias, no como objetos nuevos. Ambas instancias (original y clon) comparten la misma lista subyacente.

263. Respuesta correcta: A) SecureRandom usa fuentes de aleatoriedad criptográficamente más fuertes, dificultando que un atacante prediga los valores generados

java.util.Random usa un generador determinista predecible si se conoce la semilla; SecureRandom está diseñado para usos criptográficos, generando valores mucho más difíciles de predecir.

264. Respuesta correcta: A) Los errores pasan desapercibidos, ocultando fallos reales (incluidos posibles problemas de seguridad) y dificultando la depuración

Capturar una excepción sin registrarla ni manejarla ('swallow') oculta problemas reales, dificultando enormemente la detección y depuración de errores, incluidos posibles fallos de seguridad.

265. Respuesta correcta: A) Un ataque de 'path traversal' (por ejemplo, usando '../etc/passwd') que intente acceder a archivos fuera del directorio permitido

normalize() resuelve secuencias como '..' en la ruta, y verificar que el resultado siga comenzando dentro del directorio permitido previene que un atacante escape del directorio esperado usando path traversal.

266. Respuesta correcta: A) Las credenciales quedan expuestas en el código fuente y en el control de versiones, siendo fácilmente descubiertas si el código se filtra

Las credenciales embebidas en el código fuente quedan visibles para cualquiera con acceso al repositorio o al binario compilado. Se recomienda usar variables de entorno, gestores de secretos o archivos de configuración externos protegidos.

267. Respuesta correcta: A) Reduce la posibilidad de uso indebido o modificación accidental desde código que no debería tener acceso a esa variable

Limitar el ámbito de una variable al mínimo necesario reduce la superficie donde puede ser leída o modificada incorrectamente, facilitando también el razonamiento sobre el código.

268. Respuesta correcta: A) Validar los límites de un índice antes de acceder al array, evitando `ArrayIndexOutOfBoundsException` y posibles fallos explotables

Validar explícitamente los límites de un índice antes de usarlo previene excepciones en tiempo de ejecución y posibles comportamientos indebidos derivados de entradas no controladas.

11. Aplicaciones modulares (JPMS)

269. Respuesta correcta: A) Que otros módulos puedan acceder a las clases públicas del paquete indicado

'exports paquete' hace que los tipos públicos de ese paquete sean accesibles desde otros módulos que declaren 'requires' sobre este módulo. Sin 'exports', el paquete permanece encapsulado (fuertemente encapsulado) aunque sus clases sean públicas.

270. Respuesta correcta: A) 'requires transitive' propaga la dependencia a los módulos que a su vez dependan del módulo actual

'requires transitive' indica que cualquier módulo que dependa del módulo actual (con 'requires') automáticamente también tendrá acceso al módulo requerido de forma transitiva, sin tener que declararlo explícitamente.

271. Respuesta correcta: A) Para permitir acceso por reflexión profunda (deep reflection) a un paquete específico, solo desde el módulo indicado

'opens paquete to modulo' permite acceso reflexivo (usado comúnmente por frameworks como Hibernate o Jackson) al paquete indicado, mas restringido solo al módulo especificado, en lugar de abrirlo a todos.

272. Respuesta correcta: A) `java --module-path mods -m com.miapp/com.miapp.Main`

Para ejecutar una app modular se usa `--module-path` (o `-p`) para indicar dónde están los módulos, y `-m` módulo/clasePrincipal para indicar qué módulo y clase principal ejecutar.

273. Respuesta correcta: A) El módulo implícito al que pertenece todo el código en el classpath tradicional (sin `module-info.java`)

El código que se ejecuta desde el classpath clásico (sin declarar `module-info.java`) pertenece al 'módulo no nombrado', que puede leer todos los demás módulos pero no puede ser referenciado explícitamente por nombre desde otros módulos.

274. Respuesta correcta: A) `provides ... with ...`

'provides Interfaz with ImplementacionConcreta' declara que este módulo ofrece una implementación concreta de un servicio, que puede ser descubierta por otros módulos que usen 'uses Interfaz' junto con `ServiceLoader`.

275. Respuesta correcta: A) En la raíz del árbol de código fuente del módulo (no dentro de ningún paquete)

`module-info.java` debe ubicarse en la raíz del árbol de código fuente del módulo, sin pertenecer a ningún paquete (no lleva declaración 'package').

276. Respuesta correcta: A) Puede leer (acceder a) todos los demás módulos del module path, pero ningún módulo con nombre puede declarar 'requires' sobre él explícitamente

El módulo no nombrado (código del classpath tradicional) puede leer todos los módulos con nombre, pero no puede ser referenciado por su nombre desde otros módulos, ya que no tiene uno formal.

277. Respuesta correcta: A) Un JAR sin module-info.java colocado en el module path, al que Java asigna automáticamente un nombre derivado del archivo, y que exporta todos sus paquetes

Un módulo automático permite usar JARs tradicionales (sin module-info.java) dentro de una aplicación modular, facilitando la migración gradual de bibliotecas heredadas.

278. Respuesta correcta: A) Impide el acceso no autorizado a APIs internas de un módulo que no hayan sido explícitamente exportadas, incluso si las clases son públicas

Antes de JPMS, cualquier clase pública en el classpath era accesible desde cualquier otro código. Con JPMS, incluso las clases públicas de un paquete NO exportado permanecen inaccesibles desde fuera del módulo.

279. Respuesta correcta: A) Para crear una imagen de runtime personalizada y reducida, que incluya solo los módulos necesarios para ejecutar una aplicación específica

jlink permite ensamblar una imagen de runtime a medida, incluyendo únicamente los módulos que la aplicación realmente necesita, reduciendo el tamaño de distribución.

280. Respuesta correcta: A) Para analizar las dependencias de un archivo .class o .jar, ayudando a identificar qué módulos requiere una aplicación

jdeps analiza el bytecode de clases o JARs para reportar sus dependencias, siendo especialmente útil al migrar aplicaciones legadas hacia JPMS.

281. Respuesta correcta: A) Que la dependencia es obligatoria solo en tiempo de compilación, pero opcional en tiempo de ejecución

'requires static' declara una dependencia opcional en tiempo de ejecución: el módulo se necesita para compilar, pero la aplicación puede ejecutarse sin él si esa funcionalidad no se usa.

282. Respuesta correcta: A) Sí, se pueden exportar tantos paquetes como sea necesario mediante declaraciones 'exports' separadas

Un módulo puede tener tantas declaraciones 'exports' como paquetes públicos desee exponer, cada una en su propia línea.

283. Respuesta correcta: A) La prohibición de 'paquetes divididos' (split packages), que causa un error al resolver los módulos en tiempo de lanzamiento

JPMS prohíbe los 'split packages': el mismo paquete no puede existir en más de un módulo del mismo grafo de resolución, ya que rompe la unicidad de la organización modular.

284. Respuesta correcta: A) java --list-modules

El comando 'java --list-modules' muestra todos los módulos disponibles en la instalación actual de la JDK, incluyendo los del sistema (java.base, java.sql, etc.).

285. Respuesta correcta: A) Exponer el paquete solo al módulo indicado (com.miapp.tests), manteniéndolo encapsulado para todos los demás

'exports paquete to modulo' restringe la visibilidad del paquete exportado únicamente al módulo (o módulos) indicados, manteniéndolo encapsulado para el resto.

286. Respuesta correcta: A) Declara que el módulo consume implementaciones de un servicio (Procesador), permitiendo que ServiceLoader las descubra en tiempo de ejecución

'uses Interfaz' indica que el módulo actúa como consumidor de un servicio, habilitando a ServiceLoader.load() a descubrir las implementaciones provistas por otros módulos con 'provides ... with ...'.

287. Respuesta correcta: A) No, cada paquete que se desea exportar debe declararse explícitamente y por completo

JPMS no admite comodines en 'exports'; cada paquete que se desee exponer debe declararse explícitamente con su nombre completo.

288. Respuesta correcta: A) commons.utils (derivado del nombre del archivo, normalizando guiones y quitando la versión)

Los módulos automáticos derivan su nombre del nombre del archivo JAR, normalizando caracteres no válidos (como guiones, que se convierten en puntos) y descartando la información de versión.

289. Respuesta correcta: A) No, JPMS registra la versión como metadato informativo, pero no realiza resolución de conflictos como lo haría un gestor de dependencias

JPMS almacena la versión del módulo como información descriptiva, pero no implementa un mecanismo de resolución de conflictos de versiones como Maven o Gradle.

290. Respuesta correcta: A) Usando la opción --main-class al crear el JAR con la herramienta 'jar', o declarándola en el manifiesto del módulo

La herramienta 'jar' permite especificar --main-class al crear el JAR modular, quedando registrada en su manifiesto, de modo que 'java -m nombreModulo' (sin indicar la clase) sepa qué clase ejecutar.

291. Respuesta correcta: A) Acceso a los paquetes exportados por com.miapp.modelo, de forma transitiva

'requires transitive' propaga la dependencia: cualquier módulo que dependa de com.miapp.api (como com.miapp.web) obtiene automáticamente acceso a los paquetes exportados por com.miapp.modelo.

292. Respuesta correcta: A) 'open module' abre TODOS los paquetes del módulo para reflexión profunda automáticamente, sin necesidad de declarar 'opens' para cada uno

Declarar un módulo completo como 'open' habilita acceso reflexivo profundo a TODOS sus paquetes, útil para frameworks que dependen intensamente de reflexión, a costa de perder parte del encapsulamiento fuerte.

293. Respuesta correcta: A) Se lanza InaccessibleObjectException en tiempo de ejecución

Si un módulo no declara 'opens' (ni 'open module') para el paquete correspondiente, un intento de acceso reflexivo profundo desde otro módulo lanza InaccessibleObjectException en tiempo de ejecución.

294. Respuesta correcta: A) Un error de resolución de módulos en tiempo de lanzamiento (por ejemplo, 'module not found'), antes de ejecutar el código

JPMS resuelve el grafo completo de módulos ANTES de ejecutar cualquier código de la aplicación; si falta un módulo requerido, el lanzamiento falla inmediatamente con un error de resolución.

295. Respuesta correcta: A) Detectar en tiempo de lanzamiento dependencias faltantes o conflictivas, en lugar de fallar más tarde con errores como ClassNotFoundException en tiempo de ejecución

La 'configuración confiable' de JPMS verifica el grafo de dependencias de módulos al inicio de la aplicación, detectando de forma temprana problemas que antes solo se manifestaban en tiempo de ejecución con errores difíciles de diagnosticar.